

Voluntario: Formación de galaxias a partir de sistemas solares

Santiago Vallecillos Aranda

En el presente informe se estudia la formación de galaxias a partir de sistemas solares. El modelo se comporta como un gran sistema solar cuya estrella es el agujero negro supermasivo del centro de la Vía Láctea y los planetas son en realidad sistemas solares de masa similar al del nuestro con condiciones iniciales aleatorias. Se comprobó que el sistema está en un estado estacionario y se estudió la distribución radial media de masa alrededor del agujero negro, así como el flujo de masa absorbido por el mismo a lo largo de la simulación. Asimismo, se calculó el momento de inercia medio de todo el sistema.

Índice

1. Introducción y fundamento teórico.
2. Material utilizado.
3. Código.
4. Resultados y discusión.
5. Conclusiones.

1. Introducción y fundamento teórico.

Trabajamos con un sistema compuesto por $N+1$ cuerpos, siendo uno de ellos el agujero negro supermasivo de la Vía Láctea. Estos cuerpos los consideramos sin estructura interna y los caracterizamos por su masa y por su radio. Asumimos que su movimiento se realiza en un plano y que la única interacción presente entre los cuerpos es la gravedad y las colisiones entre ellos que vamos a considerar elásticas. Por eficiencia del programa, se sugiere en el enunciado que solo haya interacción gravitatoria entre sistemas solares cercanos. Sin embargo, en pos de tener más precisión en los cálculos y rigor físico (el alcance de la interacción gravitatoria es infinito), se consideraron todas las interacciones posibles.

Como masa del agujero negro supermasivo se tomó $8.2 \cdot 10^{36}$ kg y todos los sistemas solares se tomaron con masa cercana a la de nuestro sistema solar, $1.989 \cdot 10^{30}$ kg.

Por último, nuestro modelo contó con una condición de contorno referente a la absorción de sistemas solares por el agujero negro supermasivo del centro. Cada vez que un sistema fuese absorbido, se generaría otro aleatorio a una distancia razonable del agujero negro, con una órbita cerrada a su alrededor.

El objetivo de la simulación es obtener la evolución temporal del sistema, comprobar que está en un estado estacionario, calcular la distribución radial media de la densidad de masa, el momento de inercia medio y el flujo medio de masa absorbido por el agujero

negro.

Para realizar esta simulación se utilizó el algoritmo de Verlet. Este se basa en el desarrollo de Taylor de la posición y velocidad de cada partícula,

$$\mathbf{r}(t+h) = \mathbf{r}(t) + h \frac{d\mathbf{r}(t)}{dt} + \frac{h^2}{2} \frac{d^2\mathbf{r}(t)}{dt^2} + \mathcal{O}(h^3)$$

$$\mathbf{v}(t+h) = \mathbf{v}(t) + h \frac{d\mathbf{v}(t)}{dt} + \frac{h^2}{2} \frac{d^2\mathbf{v}(t)}{dt^2} + \mathcal{O}(h^3)$$

identificamos $\mathbf{v}(t) = \frac{d\mathbf{r}(t)}{dt}$ y $\mathbf{a}(t) = \frac{d\mathbf{v}}{dt}$ y discretizamos la derivada de la aceleración, quedándonos hasta el orden h^2 en las series de Taylor:

$$\frac{d\mathbf{a}(t)}{dt} = \frac{\mathbf{a}(t+h) - \mathbf{a}(t)}{h} + \mathcal{O}(h).$$

Así, obtenemos el algoritmo de Verlet:

$$\mathbf{r}(t+h) = \mathbf{r}(t) + h\mathbf{v}(t) + \frac{h^2}{2}\mathbf{a}(t),$$

$$\mathbf{v}(t+h) = \mathbf{v}(t) + \frac{h}{2}[\mathbf{a}(t) + \mathbf{a}(t+h)].$$

2. Material utilizado.

Para realizar las simulaciones, aparte del cluster del departamento de física estadística (JOEL), se utilizó un ordenador con las siguientes especificaciones:

- Procesador: Intel (R) Core(TM) i7-10750H CPU @ 2.60GHz (2.59 GHz)
- Tarjeta gráfica: NVIDIA GeForce GTX 1650 with Max Q Design

En cuanto a software, el código se escribió íntegramente en VSCode, utilizando el lenguaje C++ para realizar la simulación y scripts de python para graficar y analizar los resultados. Asimismo, se utilizaron LLMs para agilizar la programación y la búsqueda de errores, en concreto el Copilot integrado en VSCode y Gemini.

3. Código.

Se presenta a continuación el código que se utilizó para realizar la simulación, tanto en su versión optimizada como sin optimizar. La optimización se realizó a través de OpenMP.

Programa no optimizado:

```
#include <array>
#include <cmath>
#include <fstream>
#include <iostream>
#include <vector>
```

```

#include <chrono>
#include <random>

using namespace std;

constexpr size_t kNumPlanetas = 1000;
constexpr size_t kNumCoordenadas = 2;
constexpr double kG = 6.67430e-11;
constexpr double kUnidadDistancia = 1e21; // Radio vía láctea
constexpr double kUnidadMasa = 8.2e36;
constexpr double kRadioColision = 4.5e9; // Radio planetary del
Sistema Solar.
constexpr double kRadioRegeneracion = 0.3;
constexpr double kRadioHorizonte = 0.1;
constexpr double kPi = 3.14159265358979323846;
constexpr double kRadioColisionEscalada = kRadioColision /
kUnidadDistancia;

using Vector2D = array<double, kNumCoordenadas>;
using PlanetArray = array<Vector2D, kNumPlanetas>;
using Trajectory = vector<PlanetArray>;
using EnergyArray = vector<array<double, kNumPlanetas>>;
using MassArray = array<double, kNumPlanetas>;

void leer_datos(ifstream& data, PlanetArray& x0, PlanetArray& v0,
MassArray& masa);
void reescalar(double& h, PlanetArray& x0, PlanetArray& v0,
MassArray& masa);
void deshacer_reescalado(Trajectory& x, Trajectory& v,
Trajectory& a, MassArray& masa);
bool esta_cerca_origen(const Vector2D& posicion);
void generar_en_orbita(Vector2D& x, Vector2D& v, std::mt19937_64&
rng);
void regenerar_si_cerca_origen(PlanetArray& x, PlanetArray& v,
std::mt19937_64& rng);
void calcular_siguiente_paso(const PlanetArray& x_curr, const
PlanetArray& v_curr, const PlanetArray& a_curr, PlanetArray&
x_next, PlanetArray& v_next, PlanetArray& a_next, const
MassArray& masa, double h);
void Verlet(double& t, double h, int N, const PlanetArray& x0,
const PlanetArray& v0, Trajectory& x, Trajectory& v, Trajectory&
a, const MassArray& masa, std::mt19937_64& rng, vector<double>&
tiempos, vector<double>& energias);
PlanetArray calcular_aceleraciones(const PlanetArray& posiciones,
const MassArray& masa);
double energia_total(const PlanetArray& x, const PlanetArray& v,
const MassArray& masa);
void escribir_datos(ofstream& out, const Trajectory& x);
void escribir_datos_energia(ofstream& out, const EnergyArray&
data);
void escribir_datos_periodo(ofstream& out, const array<double,
kNumPlanetas>& periodos);
void invariantes(const Trajectory& x, const Trajectory& v, const
MassArray& masa, EnergyArray& E, Trajectory& L, Trajectory& p,
EnergyArray& mod_p);
void periodos(const EnergyArray& E, const MassArray& masa,

```

```
array<double, kNumPlanetas>& periodos);
void convertir_periodo_a_dias(array<double, kNumPlanetas>&
periodos);
double choque_elastico(double v1, double v2, double m1, double
m2);
bool haycolision(const PlanetArray& posiciones, size_t i, size_t
k);

// --- CAMBIO: Modificación de firmas para adaptarlas al enfoque
multi-run ---
// En lugar de guardar en archivos dentro de la función (lo que
pisaría los datos en cada simulación),
// ahora devuelven los valores calculados en cada simulación para
poder promediarlos en el main.
void calcular_distribucion_radial(const Trajectory& x, const
MassArray& masa, vector<double>& densidad_anillos);
double calcular_momento_inercia_medio(const Trajectory& x, const
MassArray& masa);
double calcular_flujo_masa_absorbido(const Trajectory& x, const
MassArray& masa, double h_reducido);

int main() {
    auto inicio = chrono::high_resolution_clock::now();

    ifstream data("condiciones_iniciales.txt");
    if (!data) {
        cerr << "Error: no se pudo abrir
condiciones_iniciales.txt\n";
        return 1;
    }

    ofstream trayectorias("posiciones_planetas.dat");
    if (!trayectorias) {
        cerr << "Error: no se pudo crear
posiciones_planetas.dat\n";
        return 1;
    }

    ofstream velocidades("velocidades_planetas.dat");
    if (!velocidades) {
        cerr << "Error: no se pudo crear
velocidades_planetas.dat\n";
        return 1;
    }

    ofstream aceleraciones("aceleraciones_planetas.dat");
    if (!aceleraciones) {
        cerr << "Error: no se pudo crear
aceleraciones_planetas.dat\n";
        return 1;
    }

    ofstream momento angular("momento angular.dat");
    if (!momento angular) {
        cerr << "Error: no se pudo crear momento angular.dat\n";
        return 1;
    }
}
```

```

    }

    ofstream energia("energia.dat");
    if (!energia) {
        cerr << "Error: no se pudo crear energia.dat\n";
        return 1;
    }

    ofstream momento_lineal("momento_lineal.dat");
    if (!momento_lineal) {
        cerr << "Error: no se pudo crear momento_lineal.dat\n";
        return 1;
    }

    ofstream periodo_file("periodos.dat");
    if (!periodo_file) {
        cerr << "Error: no se pudo crear periodos.dat\n";
        return 1;
    }

    ofstream estado_estacionario("estado_estacionario.dat");
    if (!estado_estacionario) {
        cerr << "Error: no se pudo crear
estado_estacionario.dat\n";
        return 1;
    }

    constexpr int N_final = 10000;
    double h = 3.156e13; // 1 millón de años en segundos

    PlanetArray x0{};
    PlanetArray v0{};
    MassArray masa{};

    leer_datos(data, x0, v0, masa);
    reescalar(h, x0, v0, masa);

    // El generador RNG se inicializa fuera una única vez con la
    // semilla temporal. Al pasarse
    // por referencia en las llamadas internas, continuará su
    // secuencia garantizando aleatoriedad e independencia.
    std::mt19937_64 rng(static_cast<unsigned
long>(chrono::high_resolution_clock::now().time_since_epoch().count(

    // --- CAMBIO: Definición de variables estructurales del
    // enfoque Multi-Run ---
    constexpr size_t kNumRuns = 10; // Número de experimentos
    independientes. Puedes subirlo a 30 o 50 para mayor precisión.
    constexpr size_t kNumAnillos = 100;

    // Contenedores estadísticos para guardar los resultados
    individuales de cada ejecución independiente
    vector<double> v_momento_inercia(kNumRuns, 0.0);
    vector<double> v_flujo_masa(kNumRuns, 0.0);
    vector<vector<double>> v_densidad_radial(kNumRuns,
vector<double>(kNumAnillos, 0.0));

```

```

// --- CAMBIO: Envoltura de la simulación en el bucle Multi-
Run ---
for (size_t run = 0; run < kNumRuns; ++run) {

    // Es indispensable crear una copia limpia de la masa
    escalada, dado que la función
    // 'deshacer_reescalado' la multiplica in-place por
    kUnidadMasa al final de cada simulación.
    MassArray masa_run = masa;
    double t_run = 0.0; // El tiempo cinemático debe
    reiniciarse en cada run

    PlanetArray x_curr = x0;
    PlanetArray v_curr = v0;

    regenerar_si_cerca_origen(x_curr, v_curr, rng);
    PlanetArray a_curr = calcular_aceleraciones(x_curr,
masa_run);
    PlanetArray x_next{};
    PlanetArray v_next{};
    PlanetArray a_next{};

    double energia_anterior = energia_total(x_curr, v_curr,
masa_run);
    constexpr double kEnergiaUmbral = 1e-5;
    constexpr int kPasosEstablesRequeridos = 10;
    int pasos_estables = 0;
    int iteracion = 0;

    // Bucle de estabilización stocástica (térmica)
    individual por cada simulación
    while (pasos_estables < kPasosEstablesRequeridos) {
        regenerar_si_cerca_origen(x_curr, v_curr, rng);
        calcular_siguiete_paso(x_curr, v_curr, a_curr,
x_next, v_next, a_next, masa_run, h);

        double energia_actual = energia_total(x_next, v_next,
masa_run);
        if (fabs(energia_actual - energia_anterior) <=
kEnergiaUmbral) {
            pasos_estables++;
        } else {
            pasos_estables = 0;
        }

        energia_anterior = energia_actual;
        x_curr = x_next;
        v_curr = v_next;
        a_curr = a_next;
        t_run += h;
        iteracion++;
    }

    Trajectory x(N_final, PlanetArray{});
    Trajectory v(N_final, PlanetArray{});

```

```

Trajectory a(N_final, PlanetArray{});
EnergyArray E(N_final);
Trajectory L(N_final, PlanetArray{});
Trajectory p(N_final, PlanetArray{});
EnergyArray mod_p(N_final);
array<double, kNumPlanetas> periodo{};
vector<double> tiempos;
vector<double> energias;

// Ejecución de la dinámica molecular de Verlet
Verlet(t_run, h, N_final, x_curr, v_curr, x, v, a,
masa_run, rng, tiempos, energias);

// --- CAMBIO: Los volcados masivos de trayectorias e
históricos temporales se limitan al Run 0 ---
// Esto previene la saturación de espacio en disco y que
archivos secuenciales se pisen o mezclen sin sentido estadístico.
if (run == 0) {
    escribir_datos(trayectorias, x);
    escribir_datos(velocidades, v);
    escribir_datos(aceleraciones, a);
}

deshacer_reescalado(x, v, a, masa_run);

if (run == 0) {
    invariantes(x, v, masa_run, E, L, p, mod_p);
    periodos(E, masa_run, periodo);
    convertir_periodo_a_dias(periodo);
    escribir_datos_periodo(periodo_file, periodo);

    const double factorEnergia = kG * kUnidadMasa *
kUnidadMasa / kUnidadDistancia;
    const double factorTiempo = pow(kUnidadDistancia,
1.5) / sqrt(kG * kUnidadMasa);
    for (size_t j = 0; j < tiempos.size(); ++j) {
        double tiempo_real = tiempos[j] * factorTiempo;
        double energia_real = energias[j] *
factorEnergia;
        estado_estacionario << tiempo_real << " " <<
energia_real << "\n";
    }
    cout << "Iteraciones hasta estabilidad (Simulación 0
de control): " << iteracion << endl;
}

// --- CAMBIO: Extracción de métricas mediante las
funciones modificadas ---
// Almacenamos el resultado escalar o vectorial del
experimento 'run' actual en los vectores correspondientes.
calcular_distribucion_radial(x, masa_run,
v_densidad_radial[run]);
v_momento_inercia[run] =
calcular_momento_inercia_medio(x, masa_run);
v_flujo_masa[run] = calcular_flujo_masa_absorbido(x,
masa_run, h);

```

```

    }

    //
    =====
    // --- CAMBIO: ANÁLISIS ESTADÍSTICO DE ERRORES RIGUROSO
    (POST-BUCLE MULTI-RUN) ---
    // Implementación rigurosa de las fórmulas de los apuntes
    (Media, Varianza y Barras de Error)
    //
    =====

    // 1. Análisis estadístico del Momento de Inercia Medio
    double mean_inercia = 0.0;
    for (double val : v_momento_inercia) mean_inercia += val;
    mean_inercia /= static_cast<double>(kNumRuns); // Estimación
    del valor medio  $\mu$   $\simeq \bar{X}$ 

    double var_inercia = 0.0;
    for (double val : v_momento_inercia) var_inercia += (val -
    mean_inercia) * (val - mean_inercia);
    var_inercia /= static_cast<double>(kNumRuns); // Estimación
    de la varianza  $\sigma^2$  según apuntes
    double sigma_inercia = sqrt(var_inercia);
    double error_inercia = sigma_inercia /
    sqrt(static_cast<double>(kNumRuns)); // Error estándar del
    estimador de la media ( $\sigma / \sqrt{N}$ )

    // Escritura del Momento de Inercia riguroso
    ofstream out_inercia("momento_inercia.dat");
    if (out_inercia) {
        out_inercia << "Momento_Inercia_Medio(kg*m^2)
    Error_Estadistico_Estandar(kg*m^2)\n";
        out_inercia << mean_inercia << " " << error_inercia <<
    "\n";
        out_inercia.close();
    }

    // 2. Análisis estadístico del Flujo de Masa Absorbido
    double mean_flujo = 0.0;
    for (double val : v_flujo_masa) mean_flujo += val;
    mean_flujo /= static_cast<double>(kNumRuns);

    double var_flujo = 0.0;
    for (double val : v_flujo_masa) var_flujo += (val -
    mean_flujo) * (val - mean_flujo);
    var_flujo /= static_cast<double>(kNumRuns);
    double sigma_flujo = sqrt(var_flujo);
    double error_flujo = sigma_flujo /
    sqrt(static_cast<double>(kNumRuns));

    // Mostrar resumen por pantalla aplicando intervalos de
    confianza del 95.4% ( $\bar{X} \pm 2\sigma/\sqrt{N}$ )
    cout <<
    "\n===== \n";
    cout << "    ANÁLISIS ESTADÍSTICO DE ERRORES RIGUROSO (MULTI-
    RUN)  \n";

```

```

        cout <<
        "=====\n";
        cout << "Momento de Inercia Medio Global: " << mean_inercia
        << " kg*m^2\n";
        cout << "Error Estadístico Estándar (\u03c3_X): " <<
        error_inercia << " kg*m^2\n";
        cout << "Intervalo Confianza (95.4%): [" << mean_inercia -
        2.0 * error_inercia << ", " << mean_inercia + 2.0 * error_inercia
        << "] kg*m^2\n\n";

        cout << "Flujo Medio de Masa Absorbido Global: " <<
        mean_flujo << " kg/s\n";
        cout << "Error Estadístico Estándar (\u03c3_X): " <<
        error_flujo << " kg/s\n";
        cout << "Intervalo Confianza (95.4%): [" << mean_flujo - 2.0
        * error_flujo << ", " << mean_flujo + 2.0 * error_flujo << "] kg/
        s\n";
        cout <<
        "=====\n\n";

        // 3. Análisis estadístico de la Distribución Radial de
        Densidad (Cálculo por celda/anillo)
        vector<double> mean_densidad(kNumAnillos, 0.0);
        vector<double> error_densidad(kNumAnillos, 0.0);

        for (size_t i = 0; i < kNumAnillos; ++i) {
            double suma_anillo = 0.0;
            for (size_t run = 0; run < kNumRuns; ++run) {
                suma_anillo += v_densidad_radial[run][i];
            }
            mean_densidad[i] = suma_anillo /
            static_cast<double>(kNumRuns);

            double var_anillo = 0.0;
            for (size_t run = 0; run < kNumRuns; ++run) {
                var_anillo += (v_densidad_radial[run][i] -
            mean_densidad[i]) * (v_densidad_radial[run][i] -
            mean_densidad[i]);
            }
            var_anillo /= static_cast<double>(kNumRuns);
            double sigma_anillo = sqrt(var_anillo);
            error_densidad[i] = sigma_anillo /
            sqrt(static_cast<double>(kNumRuns));
        }

        // Guardado de la distribución radial con su columna de error
        para pintar Barras de Error en gnuplot/python
        ofstream out_radial("densidad_radial.dat");
        if (out_radial) {
            const double r_max = kUnidadDistancia;
            const double delta_r = r_max / kNumAnillos;
            for (size_t i = 0; i < kNumAnillos; ++i) {
                double r_interno = i * delta_r;
                double r_externo = (i + 1) * delta_r;
                double r_medio = (r_interno + r_externo) / 2.0;
                // Estructura del archivo: [Radio Medio] [Densidad

```

```

Media] [Error de la Densidad]
        out_radial << r_medio << " " << mean_densidad[i] << "
" << error_densidad[i] << "\n";
    }
    out_radial.close();
}

    auto fin = chrono::high_resolution_clock::now();
    chrono::duration<double, milli> tiempo_ejecucion = fin -
inicio;
    cout << "El código multi-run completo tardó: " <<
tiempo_ejecucion.count() << " milisegundos." << endl;

    return 0;
}

void leer_datos(ifstream& data, PlanetArray& x0, PlanetArray& v0,
MassArray& masa) {
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        data >> masa[i] >> x0[i][0] >> x0[i][1] >> v0[i][0] >>
v0[i][1];
    }
}

void reescalar(double& h, PlanetArray& x0, PlanetArray& v0,
MassArray& masa) {
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        x0[i][0] /= kUnidadDistancia;
        x0[i][1] /= kUnidadDistancia;
        v0[i][0] *= pow(kUnidadDistancia, 1.5) /
(kUnidadDistancia * sqrt(kG * kUnidadMasa));
        v0[i][1] *= pow(kUnidadDistancia, 1.5) /
(kUnidadDistancia * sqrt(kG * kUnidadMasa));
        masa[i] /= kUnidadMasa;
    }
    h *= sqrt(kG * kUnidadMasa / (kUnidadDistancia *
kUnidadDistancia * kUnidadDistancia));
}

void deshacer_reescalado(Trajectory& x, Trajectory& v,
Trajectory& a, MassArray& masa) {
    const double factorVel = 1.0 / (pow(kUnidadDistancia, 1.5) /
(kUnidadDistancia * sqrt(kG * kUnidadMasa)));
    const double factorAcel = kG * kUnidadMasa /
(kUnidadDistancia * kUnidadDistancia);

    for (auto& paso : x) {
        for (auto& planeta : paso) {
            for (auto& componente : planeta) {
                componente *= kUnidadDistancia;
            }
        }
    }
    for (auto& paso : v) {
        for (auto& planeta : paso) {
            for (auto& componente : planeta) {

```

```

        componente *= factorVel;
    }
}
for (auto& paso : a) {
    for (auto& planeta : paso) {
        for (auto& componente : planeta) {
            componente *= factorAcel;
        }
    }
}
for (auto& masa_planeta : masa) {
    masa_planeta *= kUnidadMasa;
}
}

bool esta_cerca_origen(const Vector2D& posicion) {
    return posicion[0] * posicion[0] + posicion[1] * posicion[1]
< kRadioRegeneracion * kRadioRegeneracion;
}

void generar_en_orbita(Vector2D& x, Vector2D& v, std::mt19937_64&
rng) {
    std::uniform_real_distribution<double>
dist_radio(kRadioHorizonte, 1.0);
    std::uniform_real_distribution<double> dist_angulo(0.0, 2.0 *
kPi);

    const double radio = dist_radio(rng);
    const double theta = dist_angulo(rng);
    x[0] = radio * cos(theta);
    x[1] = radio * sin(theta);

    const double velocidad_orbital = sqrt(1.0 / radio);
    v[0] = -velocidad_orbital * sin(theta);
    v[1] = velocidad_orbital * cos(theta);
}

void regenerar_si_cerca_origen(PlanetArray& x, PlanetArray& v,
std::mt19937_64& rng) {
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        if (esta_cerca_origen(x[i])) {
            generar_en_orbita(x[i], v[i], rng);
        }
    }
}

void calcular_siguiete_paso(const PlanetArray& x_curr, const
PlanetArray& v_curr, const PlanetArray& a_curr, PlanetArray&
x_next, PlanetArray& v_next, PlanetArray& a_next, const
MassArray& masa, double h) {
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        for (size_t j = 0; j < kNumCoordenadas; ++j) {
            x_next[i][j] = x_curr[i][j] + v_curr[i][j] * h + 0.5
* a_curr[i][j] * h * h;
        }
    }
}

```

```

    }

    a_next = calcular_aceleraciones(x_next, masa);

    for (size_t i = 0; i < kNumPlanetas; ++i) {
        for (size_t j = 0; j < kNumCoordenadas; ++j) {
            v_next[i][j] = v_curr[i][j] + 0.5 * (a_curr[i][j] +
a_next[i][j]) * h;
        }
    }

    for (size_t i = 0; i < kNumPlanetas; ++i) {
        for (size_t k = i + 1; k < kNumPlanetas; ++k) {
            if (haycolision(x_next, i, k)) {
                double v1_final_x = choque_elastico(v_curr[i][0],
v_curr[k][0], masa[i], masa[k]);
                double v1_final_y = choque_elastico(v_curr[i][1],
v_curr[k][1], masa[i], masa[k]);
                double v2_final_x = choque_elastico(v_curr[k][0],
v_curr[i][0], masa[k], masa[i]);
                double v2_final_y = choque_elastico(v_curr[k][1],
v_curr[i][1], masa[k], masa[i]);

                v_next[i][0] = v1_final_x;
                v_next[i][1] = v1_final_y;
                v_next[k][0] = v2_final_x;
                v_next[k][1] = v2_final_y;
            }
        }
    }
}

```

```

PlanetArray calcular_aceleraciones(const PlanetArray& posiciones,
const MassArray& masa) {
    PlanetArray aceleraciones{};

    for (size_t i = 0; i < kNumPlanetas; ++i) {
        aceleraciones[i] = {0.0, 0.0};

        const double dx_sol = posiciones[i][0];
        const double dy_sol = posiciones[i][1];
        const double dist_sol2 = dx_sol * dx_sol + dy_sol *
dy_sol;
        const double dist_sol_cubica = pow(dist_sol2, 1.5);

        if (dist_sol_cubica > 0.0) {
            aceleraciones[i][0] += -dx_sol / dist_sol_cubica;
            aceleraciones[i][1] += -dy_sol / dist_sol_cubica;
        }

        for (size_t k = 0; k < kNumPlanetas; ++k) {
            if (k == i) {
                continue;
            }

            const double dx = posiciones[i][0] - posiciones[k]

```

```
[0];
    const double dy = posiciones[i][1] - posiciones[k]
[1];
    const double dist2 = dx * dx + dy * dy;

    if (dist2 <= 0.0) {
        continue;
    }

    const double distanciaCubica = pow(dist2, 1.5);
    aceleraciones[i][0] += -masa[k] * dx /
distanciaCubica;
    aceleraciones[i][1] += -masa[k] * dy /
distanciaCubica;
    }
}

return aceleraciones;
}

void Verlet(double& t, double h, int N, const PlanetArray& x0,
const PlanetArray& v0, Trajectory& x, Trajectory& v, Trajectory&
a, const MassArray& masa, std::mt19937_64& rng, vector<double>&
tiempos, vector<double>& energias) {
    PlanetArray x_curr = x0;
    PlanetArray v_curr = v0;
    regenerar_si_cerca_origen(x_curr, v_curr, rng);
    PlanetArray a_curr = calcular_aceleraciones(x_curr, masa);
    PlanetArray x_next{};
    PlanetArray v_next{};
    PlanetArray a_next{};

    t = 0.0;
    if (N > 0) {
        x[0] = x_curr;
        v[0] = v_curr;
        a[0] = a_curr;
        tiempos.push_back(t);
        energias.push_back(energia_total(x_curr, v_curr, masa));
    }

    for (int n = 0; n + 1 < N; ++n) {
        calcular_siguiete_paso(x_curr, v_curr, a_curr, x_next,
v_next, a_next, masa, h);

        x[n + 1] = x_next;
        v[n + 1] = v_next;
        a[n + 1] = a_next;

        x_curr = x_next;
        v_curr = v_next;
        a_curr = a_next;
        t += h;

        tiempos.push_back(t);
        energias.push_back(energia_total(x_curr, v_curr, masa));
    }
}
```

```

    }
}

void escribir_datos(ofstream& out, const Trajectory& x) {
    for (const auto& paso : x) {
        for (const auto& planeta : paso) {
            out << planeta[0] << ", " << planeta[1] << '\n';
        }
        out << '\n';
    }
}

void escribir_datos_energia(ofstream& out, const EnergyArray&
data) {
    for (const auto& paso : data) {
        for (size_t i = 0; i < kNumPlanetas; ++i) {
            out << paso[i] << '\n';
        }
        out << '\n';
    }
}

void escribir_datos_periodes(ofstream& out, const array<double,
kNumPlanetas>& periodos) {
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        out << periodos[i] << '\n';
    }
}

void invariantes(const Trajectory& x, const Trajectory& v, const
MassArray& masa, EnergyArray& E, Trajectory& L, Trajectory& p,
EnergyArray& mod_p) {
    for (size_t n = 0; n < x.size(); ++n) {
        for (size_t i = 0; i < kNumPlanetas; ++i) {
            const double energia_cinetica = 0.5 * masa[i] * (v[n]
[i][0] * v[n][i][0] + v[n][i][1] * v[n][i][1]);
            double masa_sol = 2e30;
            double dist_sol = sqrt(x[n][i][0] * x[n][i][0] + x[n]
[i][1] * x[n][i][1]);
            double energia_potencial = -kG * masa_sol * masa[i] /
dist_sol;

            for (size_t k = 0; k < kNumPlanetas; ++k) {
                if (k == i) {
                    continue;
                }

                const double dx = x[n][i][0] - x[n][k][0];
                const double dy = x[n][i][1] - x[n][k][1];
                const double distancia = sqrt(dx * dx + dy * dy);
                energia_potencial += -kG * masa[i] * masa[k] /
distancia;
            }

            E[n][i] = energia_cinetica + energia_potencial;
            L[n][i][0] = masa[i] * (x[n][i][1] * v[n][i][0] -

```

```

x[n][i][0] * v[n][i][1]);
    L[n][i][1] = masa[i] * (x[n][i][0] * v[n][i][1] -
x[n][i][1] * v[n][i][0]);
    p[n][i][0] = masa[i] * v[n][i][0];
    p[n][i][1] = masa[i] * v[n][i][1];
    mod_p[n][i] = sqrt(p[n][i][0] * p[n][i][0] + p[n][i]
[1] * p[n][i][1]);
    }
}

void periodos(const EnergyArray& E, const MassArray& masa,
array<double, kNumPlanetas>& periodos) {
    array<double, kNumPlanetas> energia_media{};

    for (const auto& paso : E) {
        for (size_t i = 0; i < kNumPlanetas; ++i) {
            energia_media[i] += paso[i];
        }
    }

    for (size_t i = 0; i < kNumPlanetas; ++i) {
        energia_media[i] /= static_cast<double>(E.size());
        if (energia_media[i] >= 0.0) {
            periodos[i] = 0.0;
            continue;
        }
        const double semieje_mayor = -kG * kUnidadMasa * masa[i]
/ (2.0 * energia_media[i]);
        periodos[i] = 2.0 * kPi * pow(semieje_mayor, 1.5) /
sqrt(kG * kUnidadMasa);
    }
}

void convertir_periodo_a_dias(array<double, kNumPlanetas>&
periodos) {
    constexpr double segundos_por_dia = 86400.0;
    for (auto& periodo : periodos) {
        periodo /= segundos_por_dia;
    }
}

double energia_total(const PlanetArray& x, const PlanetArray& v,
const MassArray& masa) {
    double energia = 0.0;

    for (size_t i = 0; i < kNumPlanetas; ++i) {
        energia += 0.5 * masa[i] * (v[i][0] * v[i][0] + v[i][1] *
v[i][1]);
        double dist_sol = sqrt(x[i][0] * x[i][0] + x[i][1] * x[i]
[1]);
        if (dist_sol > 0.0) {
            energia += -masa[i] / dist_sol;
        }

        for (size_t k = i + 1; k < kNumPlanetas; ++k) {

```

```

        double dx = x[i][0] - x[k][0];
        double dy = x[i][1] - x[k][1];
        double distancia = sqrt(dx * dx + dy * dy);
        if (distancia > 0.0) {
            energia += -masa[i] * masa[k] / distancia;
        }
    }
}

return energia;
}

double choque_elastico(double v1, double v2, double m1, double
m2){
    double v1_final = (v1 * (m1 - m2) + 2 * m2 * v2) / (m1 + m2);
    return v1_final;
}

bool haycolision(const PlanetArray& posiciones, size_t i, size_t
k) {
    double dx = posiciones[i][0] - posiciones[k][0];
    double dy = posiciones[i][1] - posiciones[k][1];
    double distancia = sqrt(dx * dx + dy * dy);
    return distancia < kRadioColisionEscalada;
}

// --- CAMBIO: Modificación de la función para extraer el vector
// y no escribir a disco prematuramente ---
void calcular_distribucion_radial(const Trajectory& x, const
MassArray& masa, vector<double>& densidad_anillos) {
    constexpr size_t kNumAnillos = 100;
    const double r_max = kUnidadDistancia;
    const double delta_r = r_max / kNumAnillos;

    vector<double> masa_acumulada(kNumAnillos, 0.0);
    size_t num_pasos = x.size();

    for (size_t n = 0; n < num_pasos; ++n) {
        for (size_t i = 0; i < kNumPlanetas; ++i) {
            double r = sqrt(x[n][i][0] * x[n][i][0] + x[n][i][1]
* x[n][i][1]);
            size_t indice_anillo = static_cast<size_t>(r /
delta_r);
            if (indice_anillo < kNumAnillos) {
                masa_acumulada[indice_anillo] += masa[i];
            }
        }
    }

    // Cargamos los datos calculados en el vector recibido por
    // referencia para que el main los procese estadísticamente
    densidad_anillos.assign(kNumAnillos, 0.0);
    for (size_t i = 0; i < kNumAnillos; ++i) {
        double r_interno = i * delta_r;
        double r_externo = (i + 1) * delta_r;
        double area_anillo = kPi * (r_externo * r_externo -

```

```

    r_interno * r_interno);
        double masa_promedio = masa_acumulada[i] /
static_cast<double>(num_pasos);
        densidad_anillos[i] = masa_promedio / area_anillo;
    }
}

// --- CAMBIO: Cambiado el tipo de retorno a double para extraer
// la métrica de esta simulación ---
double calcular_momento_inercia_medio(const Trajectory& x, const
MassArray& masa) {
    double inercia_total_acumulada = 0.0;
    size_t num_pasos = x.size();

    for (size_t n = 0; n < num_pasos; ++n) {
        double inercia_paso = 0.0;
        for (size_t i = 0; i < kNumPlanetas; ++i) {
            double r2 = x[n][i][0] * x[n][i][0] + x[n][i][1] *
x[n][i][1];
            inercia_paso += masa[i] * r2;
        }
        inercia_total_acumulada += inercia_paso;
    }

    // Devolvemos el escalar promedio temporal de esta simulación
    aislada
    return inercia_total_acumulada /
static_cast<double>(num_pasos);
}

// --- CAMBIO: Cambiado el tipo de retorno a double para extraer
// el flujo de masa de este run ---
double calcular_flujo_masa_absorbido(const Trajectory& x, const
MassArray& masa, double h_reducido) {
    double masa_absorbida_total = 0.0;
    size_t num_pasos = x.size();

    double radio_absorcion_metros = kRadioRegeneracion *
kUnidadDistancia;
    double r_abs_cuadrado = radio_absorcion_metros *
radio_absorcion_metros;

    for (size_t i = 0; i < kNumPlanetas; ++i) {
        bool estaba_dentro = (x[0][i][0] * x[0][i][0] + x[0][i]
[1] * x[0][i][1]) < r_abs_cuadrado;

        for (size_t n = 1; n < num_pasos; ++n) {
            double r2 = x[n][i][0] * x[n][i][0] + x[n][i][1] *
x[n][i][1];
            bool esta_dentro = r2 < r_abs_cuadrado;

            if (!estaba_dentro && esta_dentro) {
                masa_absorbida_total += masa[i];
            }
            estaba_dentro = esta_dentro;
        }
    }
}

```

```

    }

    const double factorTiempo = pow(kUnidadDistancia, 1.5) /
sqrt(kG * kUnidadMasa);
    double tiempo_total_real_segundos =
static_cast<double>(num_pasos) * h_reducido * factorTiempo;

    double flujo_medio = 0.0;
    if (tiempo_total_real_segundos > 0.0) {
        flujo_medio = masa_absorbida_total /
tiempo_total_real_segundos;
    }

    // Retornamos el flujo medio calculado exclusivamente para
este run
    return flujo_medio;
}

```

Programa optimizado:

```

#include <array>
#include <cmath>
#include <fstream>
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <omp.h>

using namespace std;

constexpr size_t kNumPlanetas = 1000;
constexpr size_t kNumCoordenadas = 2;
constexpr double kG = 6.67430e-11;
constexpr double kUnidadDistancia = 1e21; // Radio vía láctea
constexpr double kUnidadMasa = 8.2e36;
constexpr double kRadioColision = 4.5e9; // Radio planetary del
Sistema Solar.
constexpr double kRadioRegeneracion = 0.3;
constexpr double kRadioHorizonte = 0.1;
constexpr double kPi = 3.14159265358979323846;
constexpr double kRadioColisionEscalada = kRadioColision /
kUnidadDistancia;

using Vector2D = array<double, kNumCoordenadas>;
using PlanetArray = array<Vector2D, kNumPlanetas>;
using Trajectory = vector<PlanetArray>;
using EnergyArray = vector<array<double, kNumPlanetas>>;
using MassArray = array<double, kNumPlanetas>;

void leer_datos(ifstream& data, PlanetArray& x0, PlanetArray& v0,
MassArray& masa);
void reescalar(double& h, PlanetArray& x0, PlanetArray& v0,
MassArray& masa);
void deshacer_reescalado(Trajectory& x, Trajectory& v,
Trajectory& a, MassArray& masa);
bool esta_cerca_origen(const Vector2D& posicion);

```

```

void generar_en_orbita(Vector2D& x, Vector2D& v, std::mt19937_64&
rng);
void regenerar_si_cerca_origen(PlanetArray& x, PlanetArray& v,
std::mt19937_64& rng);
void calcular_siguiente_paso(const PlanetArray& x_curr, const
PlanetArray& v_curr, const PlanetArray& a_curr, PlanetArray&
x_next, PlanetArray& v_next, PlanetArray& a_next, const
MassArray& masa, double h);
void Verlet(double& t, double h, int N, const PlanetArray& x0,
const PlanetArray& v0, Trajectory& x, Trajectory& v, Trajectory&
a, const MassArray& masa, std::mt19937_64& rng, vector<double>&
tiempos, vector<double>& energias);
PlanetArray calcular_aceleraciones(const PlanetArray& posiciones,
const MassArray& masa);
double energia_total(const PlanetArray& x, const PlanetArray& v,
const MassArray& masa);
void escribir_datos(ofstream& out, const Trajectory& x);
void escribir_datos_energia(ofstream& out, const EnergyArray&
data);
void escribir_datos_periodo(ofstream& out, const array<double,
kNumPlanetas>& periodos);
void invariantes(const Trajectory& x, const Trajectory& v, const
MassArray& masa, EnergyArray& E, Trajectory& L, Trajectory& p,
EnergyArray& mod_p);
void periodos(const EnergyArray& E, const MassArray& masa,
array<double, kNumPlanetas>& periodos);
void convertir_periodo_a_dias(array<double, kNumPlanetas>&
periodos);
double choque_elastico(double v1, double v2, double m1, double
m2);
bool haycolision(const PlanetArray& posiciones, size_t i, size_t
k);

void calcular_distribucion_radial(const Trajectory& x, const
MassArray& masa, vector<double>& densidad_anillos);
double calcular_momento_inercia_medio(const Trajectory& x, const
MassArray& masa);
double calcular_flujo_masa_absorbido(const Trajectory& x, const
MassArray& masa, double h_reducido);

int main() {
    auto inicio = chrono::high_resolution_clock::now();

    ifstream data("condiciones_iniciales.txt");
    if (!data) {
        cerr << "Error: no se pudo abrir
condiciones_iniciales.txt\n";
        return 1;
    }

    ofstream trayectorias("posiciones_planetas.dat");
    if (!trayectorias) {
        cerr << "Error: no se pudo crear
posiciones_planetas.dat\n";
        return 1;
    }
}

```

```
    ofstream velocidades("velocidades_planetas.dat");
    if (!velocidades) {
        cerr << "Error: no se pudo crear
velocidades_planetas.dat\n";
        return 1;
    }

    ofstream aceleraciones("aceleraciones_planetas.dat");
    if (!aceleraciones) {
        cerr << "Error: no se pudo crear
aceleraciones_planetas.dat\n";
        return 1;
    }

    ofstream momento_angular("momento_angular.dat");
    if (!momento_angular) {
        cerr << "Error: no se pudo crear momento_angular.dat\n";
        return 1;
    }

    ofstream energia("energia.dat");
    if (!energia) {
        cerr << "Error: no se pudo crear energia.dat\n";
        return 1;
    }

    ofstream momento_lineal("momento_lineal.dat");
    if (!momento_lineal) {
        cerr << "Error: no se pudo crear momento_lineal.dat\n";
        return 1;
    }

    ofstream periodo_file("periodos.dat");
    if (!periodo_file) {
        cerr << "Error: no se pudo crear periodos.dat\n";
        return 1;
    }

    ofstream estado_estacionario("estado_estacionario.dat");
    if (!estado_estacionario) {
        cerr << "Error: no se pudo crear
estado_estacionario.dat\n";
        return 1;
    }

    constexpr int N_final = 10000;
    double h = 3.156e13; // 1 millón de años en segundos

    PlanetArray x0{};
    PlanetArray v0{};
    MassArray masa{};

    leer_datos(data, x0, v0, masa);
    reescalar(h, x0, v0, masa);
```

```

std::mt19937_64 rng(static_cast<unsigned
long>(chrono::high_resolution_clock::now().time_since_epoch().count(

constexpr size_t kNumRuns = 10;
constexpr size_t kNumAnillos = 100;

vector<double> v_momento_inercia(kNumRuns, 0.0);
vector<double> v_flujo_masa(kNumRuns, 0.0);
vector<vector<double>> v_densidad_radial(kNumRuns,
vector<double>(kNumAnillos, 0.0));

for (size_t run = 0; run < kNumRuns; ++run) {
    MassArray masa_run = masa;
    double t_run = 0.0;

    PlanetArray x_curr = x0;
    PlanetArray v_curr = v0;

    regenerar_si_cerca_origen(x_curr, v_curr, rng);
    PlanetArray a_curr = calcular_aceleraciones(x_curr,
masa_run);
    PlanetArray x_next{};
    PlanetArray v_next{};
    PlanetArray a_next{};

    double energia_anterior = energia_total(x_curr, v_curr,
masa_run);
    constexpr double kEnergiaUmbral = 1e-5;
    constexpr int kPasosEstablesRequeridos = 10;
    int pasos_estables = 0;
    int iteracion = 0;

    while (pasos_estables < kPasosEstablesRequeridos) {
        regenerar_si_cerca_origen(x_curr, v_curr, rng);
        calcular_siguiete_paso(x_curr, v_curr, a_curr,
x_next, v_next, a_next, masa_run, h);

        double energia_actual = energia_total(x_next, v_next,
masa_run);
        if (fabs(energia_actual - energia_anterior) <=
kEnergiaUmbral) {
            pasos_estables++;
        } else {
            pasos_estables = 0;
        }

        energia_anterior = energia_actual;
        x_curr = x_next;
        v_curr = v_next;
        a_curr = a_next;
        t_run += h;
        iteracion++;
    }

    Trajectory x(N_final, PlanetArray{});
    Trajectory v(N_final, PlanetArray{});

```

```

Trajectory a(N_final, PlanetArray{});
EnergyArray E(N_final);
Trajectory L(N_final, PlanetArray{});
Trajectory p(N_final, PlanetArray{});
EnergyArray mod_p(N_final);
array<double, kNumPlanetas> periodo{};
vector<double> tiempos;
vector<double> energias;

Verlet(t_run, h, N_final, x_curr, v_curr, x, v, a,
masa_run, rng, tiempos, energias);

if (run == 0) {
    escribir_datos(trayectorias, x);
    escribir_datos(velocidades, v);
    escribir_datos(aceleraciones, a);
}

deshacer_reescalado(x, v, a, masa_run);

if (run == 0) {
    invariantes(x, v, masa_run, E, L, p, mod_p);
    periodos(E, masa_run, periodo);
    convertir_periodo_a_dias(periodo);
    escribir_datos_periodo(periodo_file, periodo);

    const double factorEnergia = kG * kUnidadMasa *
kUnidadMasa / kUnidadDistancia;
    const double factorTiempo = pow(kUnidadDistancia,
1.5) / sqrt(kG * kUnidadMasa);
    for (size_t j = 0; j < tiempos.size(); ++j) {
        double tiempo_real = tiempos[j] * factorTiempo;
        double energia_real = energias[j] *
factorEnergia;
        estado_estacionario << tiempo_real << " " <<
energia_real << "\n";
    }
    cout << "Iteraciones hasta estabilidad (Simulación 0
de control): " << iteracion << endl;
}

    calcular_distribucion_radial(x, masa_run,
v_densidad_radial[run]);
    v_momento_inercia[run] =
calcular_momento_inercia_medio(x, masa_run);
    v_flujo_masa[run] = calcular_flujo_masa_absorbido(x,
masa_run, h);
}

double mean_inercia = 0.0;
for (double val : v_momento_inercia) mean_inercia += val;
mean_inercia /= static_cast<double>(kNumRuns);

double var_inercia = 0.0;
for (double val : v_momento_inercia) var_inercia += (val -
mean_inercia) * (val - mean_inercia);

```

```

var_inercia /= static_cast<double>(kNumRuns);
double sigma_inercia = sqrt(var_inercia);
double error_inercia = sigma_inercia /
sqrt(static_cast<double>(kNumRuns));

ofstream out_inercia("momento_inercia.dat");
if (out_inercia) {
    out_inercia << "Momento_Inercia_Medio(kg*m^2)
Error_Estadistico_Estandar(kg*m^2)\n";
    out_inercia << mean_inercia << " " << error_inercia <<
"\n";
    out_inercia.close();
}

double mean_flujo = 0.0;
for (double val : v_flujo_masa) mean_flujo += val;
mean_flujo /= static_cast<double>(kNumRuns);

double var_flujo = 0.0;
for (double val : v_flujo_masa) var_flujo += (val -
mean_flujo) * (val - mean_flujo);
var_flujo /= static_cast<double>(kNumRuns);
double sigma_flujo = sqrt(var_flujo);
double error_flujo = sigma_flujo /
sqrt(static_cast<double>(kNumRuns));

cout <<
"\n===== \n";
cout << "    ANÁLISIS ESTADÍSTICO DE ERRORES RIGUROSO (MULTI-
RUN)  \n";
cout <<
"===== \n";
cout << "Momento de Inercia Medio Global: " << mean_inercia
<< " kg*m^2\n";
cout << "Error Estadístico Estándar ( $\sigma_X$ ): " << error_inercia
<< " kg*m^2\n";
cout << "Intervalo Confianza (95.4%): [" << mean_inercia -
2.0 * error_inercia << ", " << mean_inercia + 2.0 * error_inercia
<< "] kg*m^2\n\n";

cout << "Flujo Medio de Masa Absorbido Global: " <<
mean_flujo << " kg/s\n";
cout << "Error Estadístico Estándar ( $\sigma_X$ ): " << error_flujo
<< " kg/s\n";
cout << "Intervalo Confianza (95.4%): [" << mean_flujo - 2.0
* error_flujo << ", " << mean_flujo + 2.0 * error_flujo << "] kg/
s\n";
cout <<
"===== \n\n";

vector<double> mean_densidad(kNumAnillos, 0.0);
vector<double> error_densidad(kNumAnillos, 0.0);

for (size_t i = 0; i < kNumAnillos; ++i) {
    double suma_anillo = 0.0;
    for (size_t run = 0; run < kNumRuns; ++run) {

```

```

        suma_anillo += v_densidad_radial[run][i];
    }
    mean_densidad[i] = suma_anillo /
static_cast<double>(kNumRuns);

    double var_anillo = 0.0;
    for (size_t run = 0; run < kNumRuns; ++run) {
        var_anillo += (v_densidad_radial[run][i] -
mean_densidad[i]) * (v_densidad_radial[run][i] -
mean_densidad[i]);
    }
    var_anillo /= static_cast<double>(kNumRuns);
    double sigma_anillo = sqrt(var_anillo);
    error_densidad[i] = sigma_anillo /
sqrt(static_cast<double>(kNumRuns));
}

ofstream out_radial("densidad_radial.dat");
if (out_radial) {
    const double r_max = kUnidadDistancia;
    const double delta_r = r_max / kNumAnillos;
    for (size_t i = 0; i < kNumAnillos; ++i) {
        double r_interno = i * delta_r;
        double r_externo = (i + 1) * delta_r;
        double r_medio = (r_interno + r_externo) / 2.0;
        out_radial << r_medio << " " << mean_densidad[i] << "
" << error_densidad[i] << "\n";
    }
    out_radial.close();
}

auto fin = chrono::high_resolution_clock::now();
chrono::duration<double, milli> tiempo_ejecucion = fin -
inicio;
cout << "El código multi-run completo tardó: " <<
tiempo_ejecucion.count() << " milisegundos." << endl;

return 0;
}

void leer_datos(ifstream& data, PlanetArray& x0, PlanetArray& v0,
MassArray& masa) {
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        data >> masa[i] >> x0[i][0] >> x0[i][1] >> v0[i][0] >>
v0[i][1];
    }
}

void reescalar(double& h, PlanetArray& x0, PlanetArray& v0,
MassArray& masa) {
#pragma omp parallel for schedule(static)
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        x0[i][0] /= kUnidadDistancia;
        x0[i][1] /= kUnidadDistancia;
        v0[i][0] *= pow(kUnidadDistancia, 1.5) /
(kUnidadDistancia * sqrt(kG * kUnidadMasa));
    }
}

```

```

        v0[i][1] *= pow(kUnidadDistancia, 1.5) /
(kUnidadDistancia * sqrt(kG * kUnidadMasa));
        masa[i] /= kUnidadMasa;
    }
    h *= sqrt(kG * kUnidadMasa / (kUnidadDistancia *
kUnidadDistancia));
}

void deshacer_reescalado(Trajectory& x, Trajectory& v,
Trajectory& a, MassArray& masa) {
    const double factorVel = 1.0 / (pow(kUnidadDistancia, 1.5) /
(kUnidadDistancia * sqrt(kG * kUnidadMasa)));
    const double factorAcel = kG * kUnidadMasa /
(kUnidadDistancia * kUnidadDistancia);

#pragma omp parallel for schedule(dynamic)
    for (size_t n = 0; n < x.size(); ++n) {
        for (size_t i = 0; i < kNumPlanetas; ++i) {
            x[n][i][0] *= kUnidadDistancia;
            x[n][i][1] *= kUnidadDistancia;
            v[n][i][0] *= factorVel;
            v[n][i][1] *= factorVel;
            a[n][i][0] *= factorAcel;
            a[n][i][1] *= factorAcel;
        }
    }

#pragma omp parallel for schedule(static)
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        masa[i] *= kUnidadMasa;
    }
}

bool esta_cerca_origen(const Vector2D& posicion) {
    return posicion[0] * posicion[0] + posicion[1] * posicion[1]
< kRadioRegeneracion * kRadioRegeneracion;
}

void generar_en_orbita(Vector2D& x, Vector2D& v, std::mt19937_64&
rng) {
    std::uniform_real_distribution<double>
dist_radio(kRadioHorizonte, 1.0);
    std::uniform_real_distribution<double> dist_angulo(0.0, 2.0 *
kPi);

    const double radio = dist_radio(rng);
    const double theta = dist_angulo(rng);
    x[0] = radio * cos(theta);
    x[1] = radio * sin(theta);

    const double velocidad_orbital = sqrt(1.0 / radio);
    v[0] = -velocidad_orbital * sin(theta);
    v[1] = velocidad_orbital * cos(theta);
}

void regenerar_si_cerca_origen(PlanetArray& x, PlanetArray& v,

```

```

std::mt19937_64& rng) {
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        if (esta_cerca_origen(x[i])) {
            generar_en_orbita(x[i], v[i], rng);
        }
    }
}

void calcular_siguiente_paso(const PlanetArray& x_curr, const
PlanetArray& v_curr, const PlanetArray& a_curr, PlanetArray&
x_next, PlanetArray& v_next, PlanetArray& a_next, const
MassArray& masa, double h) {
#pragma omp parallel for schedule(dynamic)
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        for (size_t j = 0; j < kNumCoordenadas; ++j) {
            x_next[i][j] = x_curr[i][j] + v_curr[i][j] * h + 0.5
* a_curr[i][j] * h * h;
        }
    }

    a_next = calcular_aceleraciones(x_next, masa);

#pragma omp parallel for schedule(dynamic)
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        for (size_t j = 0; j < kNumCoordenadas; ++j) {
            v_next[i][j] = v_curr[i][j] + 0.5 * (a_curr[i][j] +
a_next[i][j]) * h;
        }
    }

    for (size_t i = 0; i < kNumPlanetas; ++i) {
        for (size_t k = i + 1; k < kNumPlanetas; ++k) {
            if (haycolision(x_next, i, k)) {
                double v1_final_x = choque_elastico(v_curr[i][0],
v_curr[k][0], masa[i], masa[k]);
                double v1_final_y = choque_elastico(v_curr[i][1],
v_curr[k][1], masa[i], masa[k]);
                double v2_final_x = choque_elastico(v_curr[k][0],
v_curr[i][0], masa[k], masa[i]);
                double v2_final_y = choque_elastico(v_curr[k][1],
v_curr[i][1], masa[k], masa[i]);

                v_next[i][0] = v1_final_x;
                v_next[i][1] = v1_final_y;
                v_next[k][0] = v2_final_x;
                v_next[k][1] = v2_final_y;
            }
        }
    }
}

PlanetArray calcular_aceleraciones(const PlanetArray& posiciones,
const MassArray& masa) {
    PlanetArray aceleraciones{};

#pragma omp parallel for schedule(dynamic)

```

```

    for (size_t i = 0; i < kNumPlanetas; ++i) {
        aceleraciones[i] = {0.0, 0.0};

        const double dx_sol = posiciones[i][0];
        const double dy_sol = posiciones[i][1];
        const double dist_sol2 = dx_sol * dx_sol + dy_sol *
dy_sol;
        const double dist_sol_cubica = pow(dist_sol2, 1.5);

        if (dist_sol_cubica > 0.0) {
            aceleraciones[i][0] += -dx_sol / dist_sol_cubica;
            aceleraciones[i][1] += -dy_sol / dist_sol_cubica;
        }

        for (size_t k = 0; k < kNumPlanetas; ++k) {
            if (k == i) {
                continue;
            }

            const double dx = posiciones[i][0] - posiciones[k]
[0];
            const double dy = posiciones[i][1] - posiciones[k]
[1];
            const double dist2 = dx * dx + dy * dy;

            if (dist2 <= 0.0) {
                continue;
            }

            const double distanciaCubica = pow(dist2, 1.5);
            aceleraciones[i][0] += -masa[k] * dx /
distanciaCubica;
            aceleraciones[i][1] += -masa[k] * dy /
distanciaCubica;
        }

        return aceleraciones;
    }

void Verlet(double& t, double h, int N, const PlanetArray& x0,
const PlanetArray& v0, Trajectory& x, Trajectory& v, Trajectory&
a, const MassArray& masa, std::mt19937_64& rng, vector<double>&
tiempos, vector<double>& energias) {
    PlanetArray x_curr = x0;
    PlanetArray v_curr = v0;
    regenerar_si_cerca_origen(x_curr, v_curr, rng);
    PlanetArray a_curr = calcular_aceleraciones(x_curr, masa);
    PlanetArray x_next{};
    PlanetArray v_next{};
    PlanetArray a_next{};

    t = 0.0;
    if (N > 0) {
        x[0] = x_curr;
        v[0] = v_curr;
    }

```

```

        a[0] = a_curr;
        tiempos.push_back(t);
        energias.push_back(energia_total(x_curr, v_curr, masa));
    }

    for (int n = 0; n + 1 < N; ++n) {
        calcular_siguiete_paso(x_curr, v_curr, a_curr, x_next,
v_next, a_next, masa, h);

        x[n + 1] = x_next;
        v[n + 1] = v_next;
        a[n + 1] = a_next;

        x_curr = x_next;
        v_curr = v_next;
        a_curr = a_next;
        t += h;
        tiempos.push_back(t);
        energias.push_back(energia_total(x_curr, v_curr, masa));
    }
}

void escribir_datos(ofstream& out, const Trajectory& x) {
    for (const auto& paso : x) {
        for (const auto& planeta : paso) {
            out << planeta[0] << ", " << planeta[1] << '\n';
        }
        out << '\n';
    }
}

void escribir_datos_energia(ofstream& out, const EnergyArray&
data) {
    for (const auto& paso : data) {
        for (size_t i = 0; i < kNumPlanetas; ++i) {
            out << paso[i] << '\n';
        }
        out << '\n';
    }
}

void escribir_datos_periodo(ofstream& out, const array<double,
kNumPlanetas>& periodos) {
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        out << periodos[i] << '\n';
    }
}

void invariantes(const Trajectory& x, const Trajectory& v, const
MassArray& masa, EnergyArray& E, Trajectory& L, Trajectory& p,
EnergyArray& mod_p) {
#pragma omp parallel for collapse(2) schedule(dynamic)
    for (size_t n = 0; n < x.size(); ++n) {
        for (size_t i = 0; i < kNumPlanetas; ++i) {
            const double energia_cinetica = 0.5 * masa[i] * (v[n]
[i][0] * v[n][i][0] + v[n][i][1] * v[n][i][1]);

```

```

        double masa_sol = 2e30;
        double dist_sol = sqrt(x[n][i][0] * x[n][i][0] + x[n]
[i][1] * x[n][i][1]);
        double energia_potencial = -kG * masa_sol * masa[i] /
dist_sol;

        for (size_t k = 0; k < kNumPlanetas; ++k) {
            if (k == i) {
                continue;
            }

            const double dx = x[n][i][0] - x[n][k][0];
            const double dy = x[n][i][1] - x[n][k][1];
            const double distancia = sqrt(dx * dx + dy * dy);
            energia_potencial += -kG * masa[i] * masa[k] /
distancia;
        }

        E[n][i] = energia_cinetica + energia_potencial;
        L[n][i][0] = masa[i] * (x[n][i][1] * v[n][i][0] -
x[n][i][0] * v[n][i][1]);
        L[n][i][1] = masa[i] * (x[n][i][0] * v[n][i][1] -
x[n][i][1] * v[n][i][0]);
        p[n][i][0] = masa[i] * v[n][i][0];
        p[n][i][1] = masa[i] * v[n][i][1];
        mod_p[n][i] = sqrt(p[n][i][0] * p[n][i][0] + p[n][i]
[1] * p[n][i][1]);
    }
}

void periodos(const EnergyArray& E, const MassArray& masa,
array<double, kNumPlanetas>& periodos) {
    array<double, kNumPlanetas> energia_media{};

    for (const auto& paso : E) {
        for (size_t i = 0; i < kNumPlanetas; ++i) {
            energia_media[i] += paso[i];
        }
    }

    for (size_t i = 0; i < kNumPlanetas; ++i) {
        energia_media[i] /= static_cast<double>(E.size());
        if (energia_media[i] >= 0.0) {
            periodos[i] = 0.0;
            continue;
        }
        const double semieje_mayor = -kG * kUnidadMasa * masa[i]
/ (2.0 * energia_media[i]);
        periodos[i] = 2.0 * kPi * pow(semieje_mayor, 1.5) /
sqrt(kG * kUnidadMasa);
    }
}

void convertir_periodo_a_dias(array<double, kNumPlanetas>&
periodos) {

```

```
constexpr double segundos_por_dia = 86400.0;
for (auto& periodo : periodos) {
    periodo /= segundos_por_dia;
}

double energia_total(const PlanetArray& x, const PlanetArray& v,
const MassArray& masa) {
    double energia = 0.0;

    for (size_t i = 0; i < kNumPlanetas; ++i) {
        energia += 0.5 * masa[i] * (v[i][0] * v[i][0] + v[i][1] *
v[i][1]);
        double dist_sol = sqrt(x[i][0] * x[i][0] + x[i][1] * x[i]
[1]);
        if (dist_sol > 0.0) {
            energia += -masa[i] / dist_sol;
        }

        for (size_t k = i + 1; k < kNumPlanetas; ++k) {
            double dx = x[i][0] - x[k][0];
            double dy = x[i][1] - x[k][1];
            double distancia = sqrt(dx * dx + dy * dy);
            if (distancia > 0.0) {
                energia += -masa[i] * masa[k] / distancia;
            }
        }
    }

    return energia;
}

double choque_elastico(double v1, double v2, double m1, double
m2){
    double v1_final = (v1 * (m1 - m2) + 2 * m2 * v2) / (m1 + m2);
    return v1_final;
}

bool haycolision(const PlanetArray& posiciones, size_t i, size_t
k) {
    double dx = posiciones[i][0] - posiciones[k][0];
    double dy = posiciones[i][1] - posiciones[k][1];
    double distancia = sqrt(dx * dx + dy * dy);
    return distancia < kRadioColisionEscalada;
}

void calcular_distribucion_radial(const Trajectory& x, const
MassArray& masa, vector<double>& densidad_anillos) {
    constexpr size_t kNumAnillos = 100;
    const double r_max = kUnidadDistancia;
    const double delta_r = r_max / kNumAnillos;

    vector<double> masa_acumulada(kNumAnillos, 0.0);
    size_t num_pasos = x.size();
    int nthreads = omp_get_max_threads();
    vector<array<double, kNumAnillos>> masa_local(nthreads);
```

```

    for (auto& arr : masa_local) {
        arr.fill(0.0);
    }

#pragma omp parallel
    {
        int tid = omp_get_thread_num();
        auto& local_acumulado = masa_local[tid];

#pragma omp for schedule(dynamic)
        for (size_t n = 0; n < num_pasos; ++n) {
            for (size_t i = 0; i < kNumPlanetas; ++i) {
                double r = sqrt(x[n][i][0] * x[n][i][0] + x[n][i]
[1] * x[n][i][1]);
                size_t indice_anillo = static_cast<size_t>(r /
delta_r);
                if (indice_anillo < kNumAnillos) {
                    local_acumulado[indice_anillo] += masa[i];
                }
            }
        }

        for (int t = 0; t < nthreads; ++t) {
            for (size_t i = 0; i < kNumAnillos; ++i) {
                masa_acumulada[i] += masa_local[t][i];
            }
        }

        densidad_anillos.assign(kNumAnillos, 0.0);
        for (size_t i = 0; i < kNumAnillos; ++i) {
            double r_interno = i * delta_r;
            double r_externo = (i + 1) * delta_r;
            double area_anillo = kPi * (r_externo * r_externo -
r_interno * r_interno);
            double masa_promedio = masa_acumulada[i] /
static_cast<double>(num_pasos);
            densidad_anillos[i] = masa_promedio / area_anillo;
        }
    }

double calcular_momento_inercia_medio(const Trajectory& x, const
MassArray& masa) {
    double inercia_total_acumulada = 0.0;
    size_t num_pasos = x.size();

#pragma omp parallel for reduction(+ : inercia_total_acumulada)
schedule(dynamic)
    for (size_t n = 0; n < num_pasos; ++n) {
        double inercia_paso = 0.0;
        for (size_t i = 0; i < kNumPlanetas; ++i) {
            double r2 = x[n][i][0] * x[n][i][0] + x[n][i][1] *
x[n][i][1];
            inercia_paso += masa[i] * r2;
        }
        inercia_total_acumulada += inercia_paso;
    }
}

```

```

    }

    return inercia_total_acumulada /
    static_cast<double>(num_pasos);
}

double calcular_flujo_masa_absorbido(const Trajectory& x, const
MassArray& masa, double h_reducido) {
    double masa_absorbida_total = 0.0;
    size_t num_pasos = x.size();

    double radio_absorcion_metros = kRadioRegeneracion *
kUnidadDistancia;
    double r_abs_cuadrado = radio_absorcion_metros *
radio_absorcion_metros;

#pragma omp parallel for reduction(+ : masa_absorbida_total)
schedule(dynamic)
    for (size_t i = 0; i < kNumPlanetas; ++i) {
        bool estaba_dentro = (x[0][i][0] * x[0][i][0] + x[0][i]
[1] * x[0][i][1]) < r_abs_cuadrado;

        for (size_t n = 1; n < num_pasos; ++n) {
            double r2 = x[n][i][0] * x[n][i][0] + x[n][i][1] *
x[n][i][1];
            bool esta_dentro = r2 < r_abs_cuadrado;

            if (!estaba_dentro && esta_dentro) {
                masa_absorbida_total += masa[i];
            }
            estaba_dentro = esta_dentro;
        }
    }

    const double factorTiempo = pow(kUnidadDistancia, 1.5) /
sqrt(kG * kUnidadMasa);
    double tiempo_total_real_segundos =
static_cast<double>(num_pasos) * h_reducido * factorTiempo;

    double flujo_medio = 0.0;
    if (tiempo_total_real_segundos > 0.0) {
        flujo_medio = masa_absorbida_total /
tiempo_total_real_segundos;
    }

    return flujo_medio;
}

```

A continuación se presentan los scripts de python utilizados para generar las animaciones del sistema y utilizados para graficar la energía en busca de comprobar el estado estacionario y la densidad radial de masa en ese orden.

```

In [ ]: # =====
# ANIMACION SISTEMA SOLAR (VERSIÓN ALTA EFICIENCIA)
# =====

```

```

from matplotlib import pyplot as plt
from matplotlib.animation import FuncAnimation
from matplotlib.patches import Circle
import numpy as np

try:
    import cupy as cp
    gpu_available = True
except ImportError:
    cp = None
    gpu_available = False

# Parámetros
# =====
file_in = "posiciones_planetas.dat"
file_out = "planetas"
use_gpu = True

# --- NUEVOS PARÁMETROS DE OPTIMIZACIÓN ---
frame_step = 100          # PROCESA 1 DE CADA X FOTOGRAMAS. Auméntalo si
max_trail_length = 50     # LONGITUD MÁXIMA DE LA ESTELA. Evita que la mem
# -----

x_min = -1
x_max = 1
y_min = -1
y_max = 1

interval = 20 # Tiempo entre fotogramas en milisegundos
show_trail = True
trail_width = 1
save_to_file = True
dpi = 150

planet_radius = 0.01
planet_colors = ["tab:blue", "tab:orange", "tab:green", "tab:red",
                 "tab:purple", "tab:brown", "tab:pink", "tab:gray"]

# Lectura del fichero de datos
# =====
print("Leyendo fichero de datos...")
with open(file_in, "r") as f:
    data_str = f.read()

xp = cp if gpu_available and use_gpu else np
if gpu_available and use_gpu:
    def to_cpu(array): return xp.asnumpy(array)
else:
    def to_cpu(array): return array

frames_data = list()

# OPTIMIZACIÓN 1: Descartar bloques de texto antes de procesarlos numéricos
bloques_texto = [b for b in data_str.split("\n\n") if b.strip()]
bloques_texto = bloques_texto[::frame_step] # Aplicar el salto de fotogra

print(f"Procesando {len(bloques_texto)} fotogramas (con salto de {frame_s

for frame_data_str in bloques_texto:

```

```

frame_data = list()
for planet_pos_str in frame_data_str.split("\n"):
    if planet_pos_str.strip():
        planet_pos = xp.fromstring(planet_pos_str, sep=",")
        if planet_pos.size > 0:
            frame_data.append(planet_pos)

if frame_data:
    frames_data.append(xp.vstack(frame_data))

nplanets = int(frames_data[0].shape[0])

# OPTIMIZACIÓN 2: Precalculando el historial en una matriz 3D para acceso i
# Dimensión: (num_fotogramas, num_planetas, coordenadas_xy)
print("Construyendo matriz histórica 3D para acelerar el renderizado...")
historial = np.array([to_cpu(f) for f in frames_data])

# Creación de la animación/gráfico
# =====
print("Generando animación...")
fig, ax = plt.subplots()
ax.axis("equal")
ax.set_xlim(x_min, x_max)
ax.set_ylim(y_min, y_max)
ax.set_facecolor('black') # Fondo negro opcional, queda más "espacial"

if not hasattr(planet_radius, "__iter__"):
    planet_radius = planet_radius*np.ones(nplanets)
else:
    if not nplanets == len(planet_radius):
        raise ValueError("El número de radios no coincide con el de plane

if not hasattr(planet_colors, "__iter__") or isinstance(planet_colors, st
    planet_colors = [planet_colors]*nplanets
elif len(planet_colors) < nplanets:
    planet_colors = (planet_colors * ((nplanets // len(planet_colors)) +

planet_points = list()
planet_trails = list()

for j_planet, (radius, color) in enumerate(zip(planet_radius, planet_colo
    x, y = historial[0, j_planet, 0], historial[0, j_planet, 1]

    planet_point = Circle((x, y), radius, facecolor=color, edgecolor="non
    ax.add_artist(planet_point)
    planet_points.append(planet_point)

    if show_trail:
        planet_trail, = ax.plot(x, y, "-", linewidth=trail_width, color=c
        planet_trails.append(planet_trail)

def update(j_frame):
    for j_planet in range(nplanets):
        x = historial[j_frame, j_planet, 0]
        y = historial[j_frame, j_planet, 1]
        planet_points[j_planet].center = (x, y)

        if show_trail:
            # OPTIMIZACIÓN 3: Extraer la estela con slicing (super rápido

```

```

        inicio_estela = max(0, j_frame - max_trail_length)
        xs_new = historial[inicio_estela:j_frame+1, j_planet, 0]
        ys_new = historial[inicio_estela:j_frame+1, j_planet, 1]
        planet_trails[j_planet].set_data(xs_new, ys_new)

    return planet_points + planet_trails

def init_anim():
    if show_trail:
        for j_planet in range(nplanets):
            planet_trails[j_planet].set_data([], [])
    return planet_points + planet_trails

nframes = len(frames_data)

if nframes > 1:
    animation = FuncAnimation(
        fig, update, init_func=init_anim,
        frames=nframes, blit=True, interval=interval)

    if save_to_file:
        animation.save("{}mp4".format(file_out), dpi=dpi)
        print(f"Vídeo guardado con éxito como {file_out}.mp4")
    else:
        plt.show()
else:
    if save_to_file:
        fig.savefig("{}pdf".format(file_out))
        print("Imagen guardada con éxito.")
    else:
        plt.show()

# Modificación del programa proporcionado por la profesora Jara Juana Ber

```

```

In [ ]: import argparse
        from pathlib import Path
        import numpy as np
        from matplotlib import pyplot as plt

        def main():
            parser = argparse.ArgumentParser(
                description="Genera una gráfica de la energía total del sistema e
            )
            parser.add_argument(
                "--input",
                default="estado_estacionario.dat",
                help="Fichero de entrada con tiempo y energía (por defecto: estad
            )
            parser.add_argument(
                "--output",
                default="energia_vs_tiempo.png",
                help="Nombre del fichero de salida de la gráfica (por defecto: en
            )
            parser.add_argument(
                "--show",
                action="store_true",
                help="Mostrar la gráfica en pantalla además de guardarla",
            )
            args = parser.parse_args()

```

```

data_path = Path(args.input)
if not data_path.exists():
    raise SystemExit(f"Error: no existe el fichero de entrada '{args.

data = np.loadtxt(data_path)
if data.ndim == 1:
    data = data.reshape(1, -1)

if data.shape[1] < 2:
    raise SystemExit("Error: el fichero de entrada debe tener al meno

tiempos = data[:, 0]
energias = data[:, 1]

plt.figure(figsize=(10, 6))
plt.plot(tiempos, energias, color="tab:blue", linewidth=1.5)
plt.title("Energía total del sistema vs. tiempo")
plt.xlabel("Tiempo [s]")
plt.ylabel("Energía total [J]")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig(args.output, dpi=150)
print(f"Gráfica guardada en '{args.output}'")

if args.show:
    plt.show()

if __name__ == "__main__":
    main()

```

```

In [ ]: import numpy as np
import matplotlib.pyplot as plt

# 1. Cargar los datos del archivo
# Columna 0: Radio medio (m)
# Columna 1: Densidad media (kg/m^2)
# Columna 2: Error estadístico de la densidad (kg/m^2)
datos = np.loadtxt("densidad_radial.dat")
r_medio = datos[:, 0]
densidad = datos[:, 1]
error = datos[:, 2]

# 2. Configurar la figura
plt.figure(figsize=(10, 6))

# 3. Dibujar la densidad con barras de error clásicas
plt.errorbar(r_medio, densidad, yerr=error,
             fmt='-o',          # Formato: línea sólida con marcadores de
             color='#1f77b4',   # Color de la línea y el punto
             ecolore='#333333',  # Color de las barras de error (gris oscu
             elinewidth=1.2,     # Grosor de la barra de error
             capsize=3,         # Tamaño del remate horizontal (la "T") d
             markersize=4,      # Tamaño del punto de los datos
             label='Densidad media con error (1$\sigma$)')

# 4. Configurar ejes y etiquetas con las unidades correctas
plt.title("Distribución Radial de Densidad del Sistema", fontsize=14, pad
plt.xlabel("Posición radial r (m)", fontsize=12)

```

```
plt.ylabel("Densidad de masa superficial  $\rho$  (kg/m2)", fontsize=12)

# Formatear ejes en notación científica (distancias del orden de 1e21)
plt.ticklabel_format(style='sci', axis='x', scilimits=(0,0))
plt.ticklabel_format(style='sci', axis='y', scilimits=(0,0))

# 5. Detalles estéticos
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(loc='upper right', fontsize=11)
plt.tight_layout()

# 6. Guardar y mostrar
plt.savefig("densidad_radial_plot_barras.png", dpi=300)
print("¡Gráfica generada y guardada como 'densidad_radial_plot_barras.png'")
plt.show()
```

4. Resultados y discusión.

1. Simulación de 1000 sistemas solares.

En primer lugar se simuló el modelo para 1000 sistemas solares orbitando en torno al agujero negro. Se realizó una simulación optimizada y una no optimizada que se comparan a continuación.

Evolución temporal y estado estacionario.

Se simularon las trayectorias de todos los sistemas solares en una animación de la que se presenta un fotograma para cada programa (optimizado y sin optimizar):

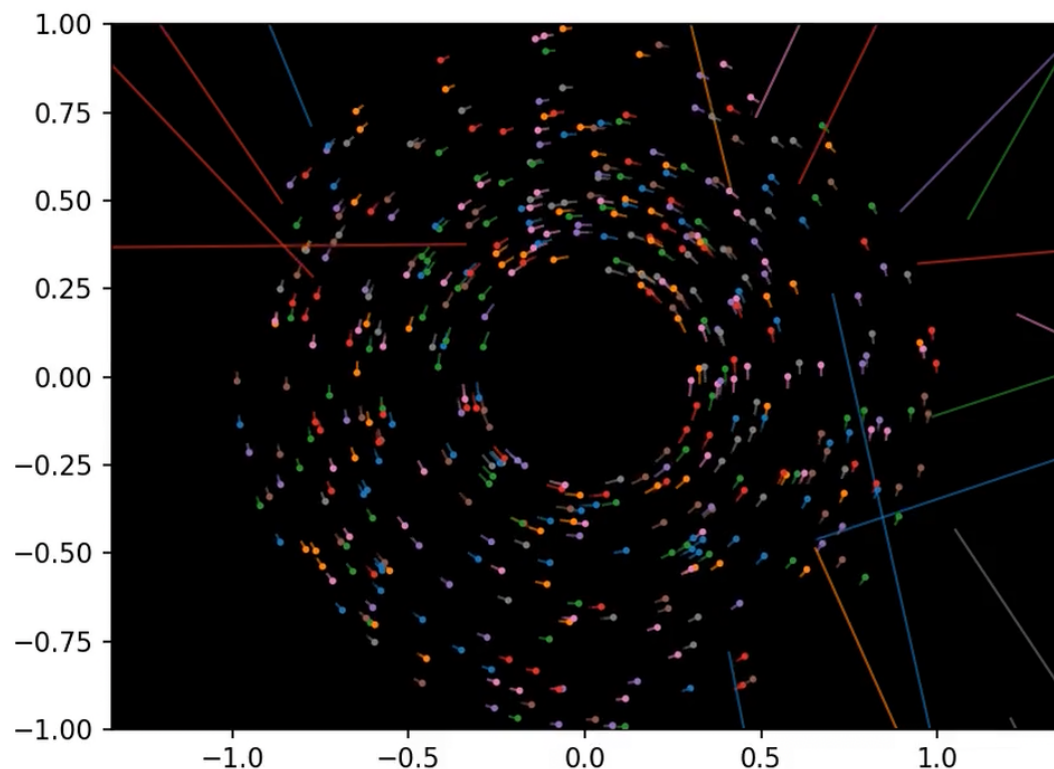


Figura 1: Fotograma de la animación de la evolución temporal del modelo al inicio de la simulación no optimizada. Se observa el escape de ciertos sistemas.

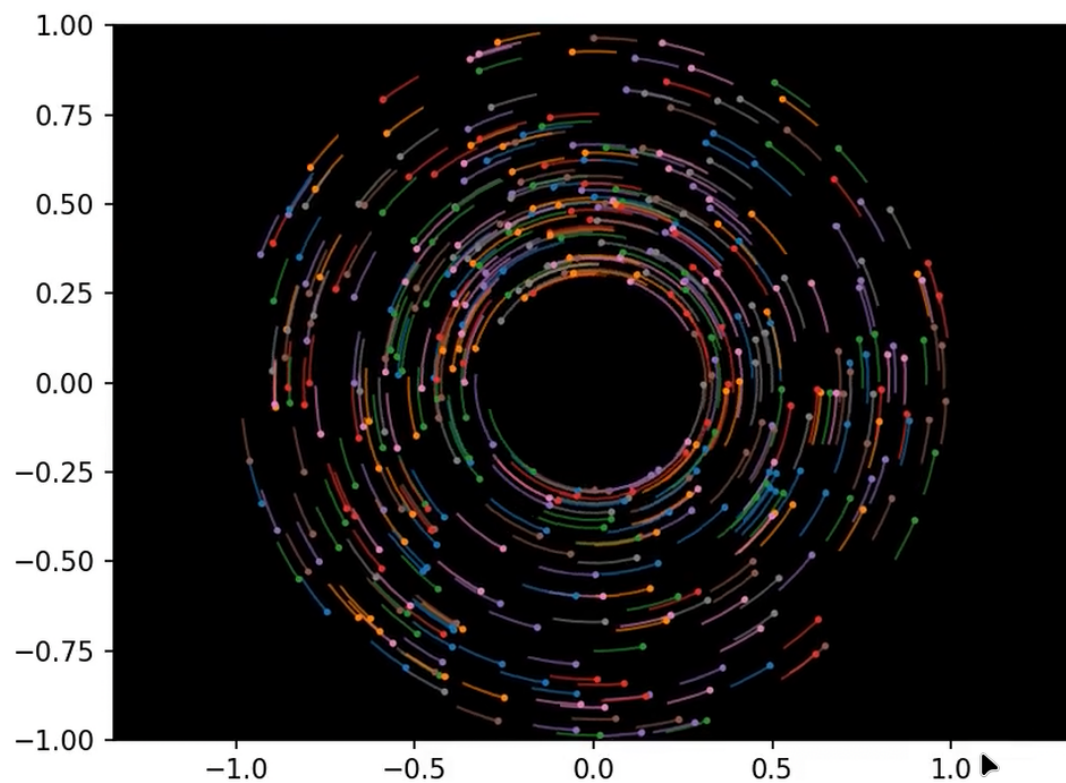


Figura 2: Fotograma de la animación de la evolución temporal del modelo al final de la simulación no optimizada. Se observan órbitas circulares y la gran densidad de sistemas en los alrededores del horizonte de los sucesos.

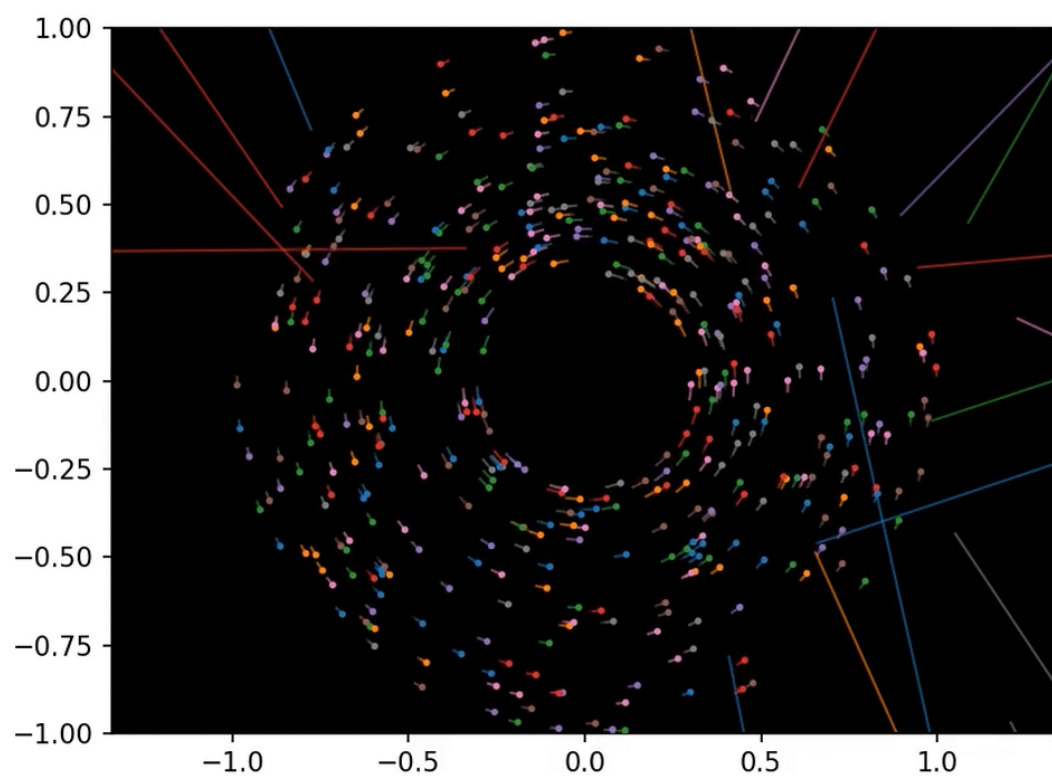


Figura 3: Fotograma de la animación de la evolución temporal del modelo al inicio de la simulación optimizada. Se observa el escape de ciertos sistemas.

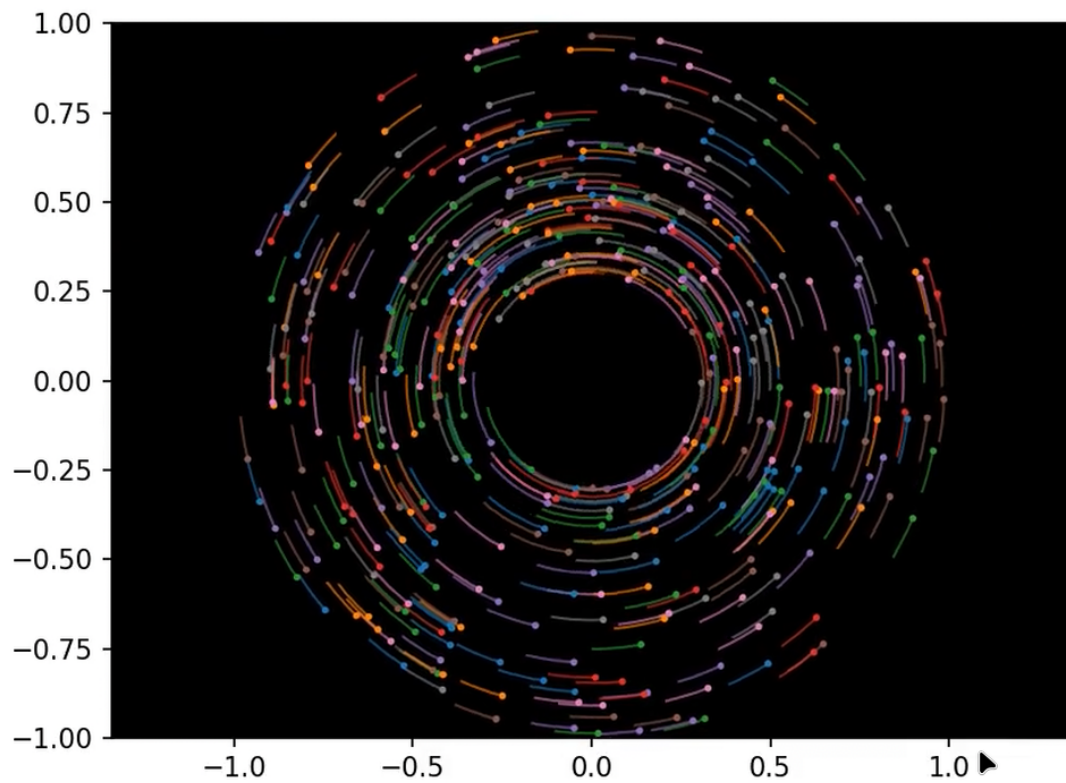


Figura 4: Fotograma de la animación de la evolución temporal del modelo al final de la simulación optimizada. Se observan órbitas circulares y la gran densidad de sistemas en los alrededores del horizonte de los sucesos.

Asimismo, se calculó la energía media para cada paso del tiempo y se graficó frente al tiempo. Los resultados obtenidos para cada simulación son los siguientes.

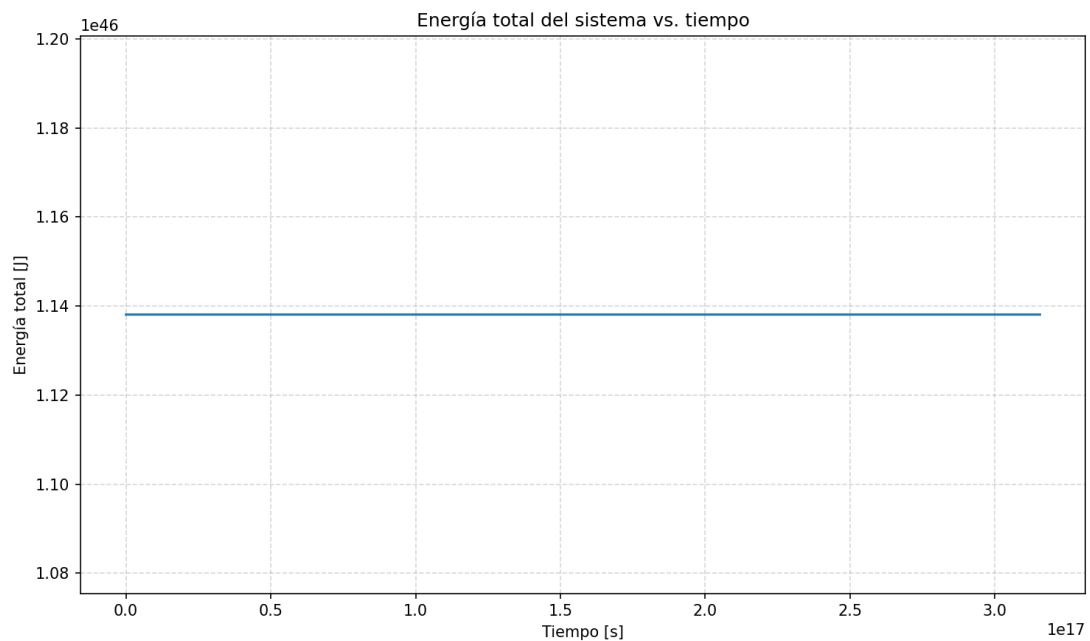


Figura 5: Energía media frente al tiempo del sistema a lo largo de la simulación no optimizada.

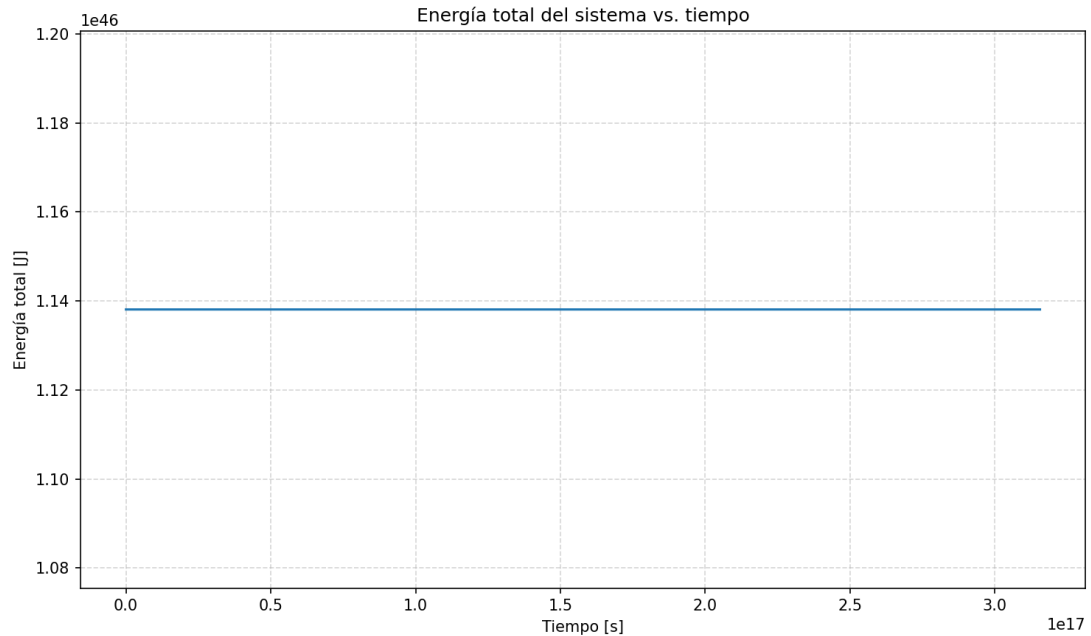


Figura 6: Energía media frente al tiempo del sistema a lo largo de la simulación optimizada.

Observamos en ambos caso que la energía media permanece constante en el tiempo, de hecho, con el mismo valor en ambos casos (lo cual es lógico pues ambas simulaciones usan las mismas condiciones iniciales). Esto nos asegura que el sistema se encuentra en un estado estacionario porque su energía no depende del tiempo.

Distribución radial media en función de la distancia.

A continuación, se graficó la distribución radial de masa media en función de la distancia al agujero negro (el centro del sistema). Para las dos simulaciones, las gráficas son las siguientes:

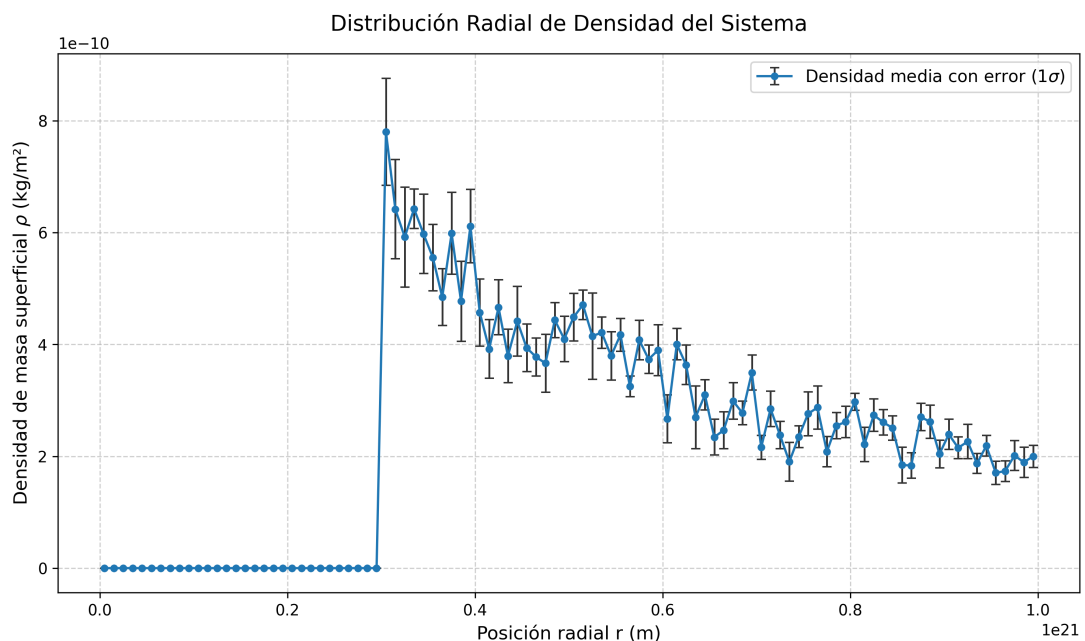


Figura 7: Distribución radial de la densidad de masa superficial en función de la distancia al origen obtenida mediante la simulación no optimizada.

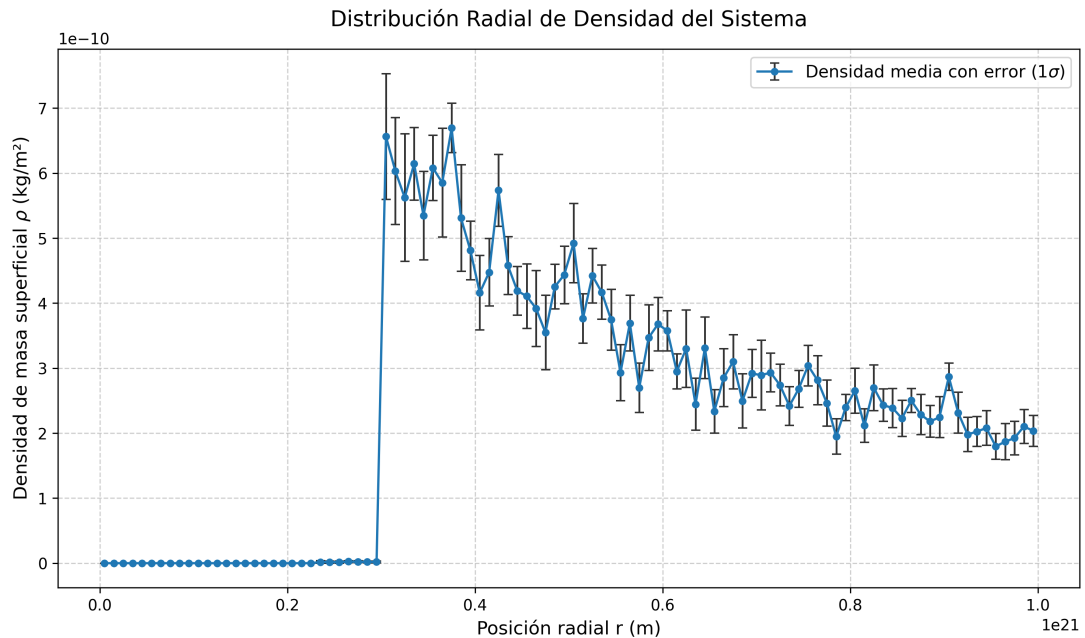


Figura 8: Distribución radial de la densidad de masa superficial en función de la distancia al origen obtenida mediante la simulación optimizada.

En este caso, sí encontramos ciertas diferenciaciones sutiles entre los resultados de ambas simulaciones. Sin embargo, las dos llegan a una conclusión similar. En las cercanías del agujero negro no hay masa, pues todos los sistemas que cruzan el horizonte de los sucesos son absorbidos por el mismo. A continuación, se puede apreciar una densidad muy alta, que representa un disco de acreción alrededor del agujero negro. Por último, conforme nos vamos alejando del origen, vamos viendo que la densidad disminuye.

Momento de inercia medio.

Para ambas simulaciones se calculó el momento de inercia medio.

Simulación sin optimizar: $M = (7.630600000 \pm 0.000000031) \cdot 10^{80} \text{ kg} \cdot \text{m}^2$

Simulación optimizada: $M = (7.630600000 \pm 0.000000026) \cdot 10^{80} \text{ kg} \cdot \text{m}^2$

La única diferenciación entre los dos programas es que la versión optimizada obtiene un error menor. Vemos que la incertidumbre es ocho órdenes de magnitud menor que el valor obtenido, lo que indica una enorme estabilidad del sistema. Asimismo, el valor obtenido es del orden de 10^{80} , lo que viene explicado en parte por la influencia de los sistemas más alejados del origen por la dependencia del momento de inercia del cuadrado de la distancia. Este cálculo se promedió de 10 simulaciones independientes.

Flujo de masa absorbido medio.

En este apartado, el programa sin optimizar dio como resultado un flujo medio de masa absorbido nulo. Esto es el resultado de carecer del rendimiento necesario para realizar múltiples ejecuciones del código. La versión secuencial ofreció un único dato aislado sujeto a una alta varianza estadística.

En cuanto a la simulación optimizada, el flujo medio de masa absorbido global fue de

$(12.1 \pm 7.7) \cdot 10^{11}$ kg/s. El alto valor del error respecto al valor medio muestra que el fenómeno de absorción de un planeta por el agujero negro es un evento discreto y poco frecuente.

Comparación de rendimiento.

La simulación optimizada tardó casi tres horas en ejecutarse por completo, mientras que la no optimizada tardó algo más de media hora más. La mejora en el rendimiento en esta práctica al aplicar OpenMP no fue muy notoria. Esto se debe a las cantidades masivas de datos que se escribieron, lo cual es un proceso estrictamente secuencial y no se puede paralelizar.

2. Simulación de 100 sistemas solares.

Para realizar la simulación también en el cluster, se simuló el mismo sistema para tan solo 100 cuerpos. En un principio se iba a realizar una simulación de 1000 planetas para realizar una comparativa, sin embargo, el tiempo de ejecución del programa fue demasiado grande y dio problemas. A continuación se realiza un análisis de los resultados análogo para este caso.

Evolución temporal y estado estacionario.

Se simularon las trayectorias de todos los sistemas solares en una animación de la que se presenta un fotograma para cada programa (optimizado y sin optimizar):

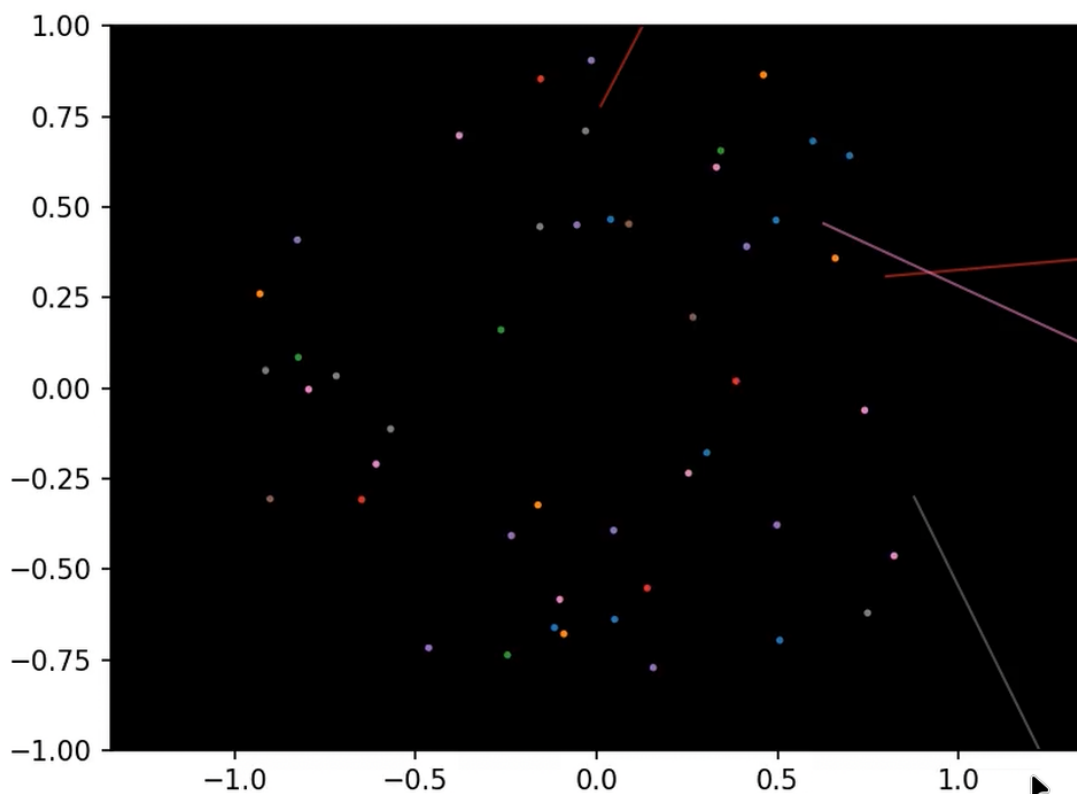


Figura 9: Fotograma de la animación de la evolución temporal del modelo al inicio de la simulación optimizada. Se observa el escape de ciertos sistemas.

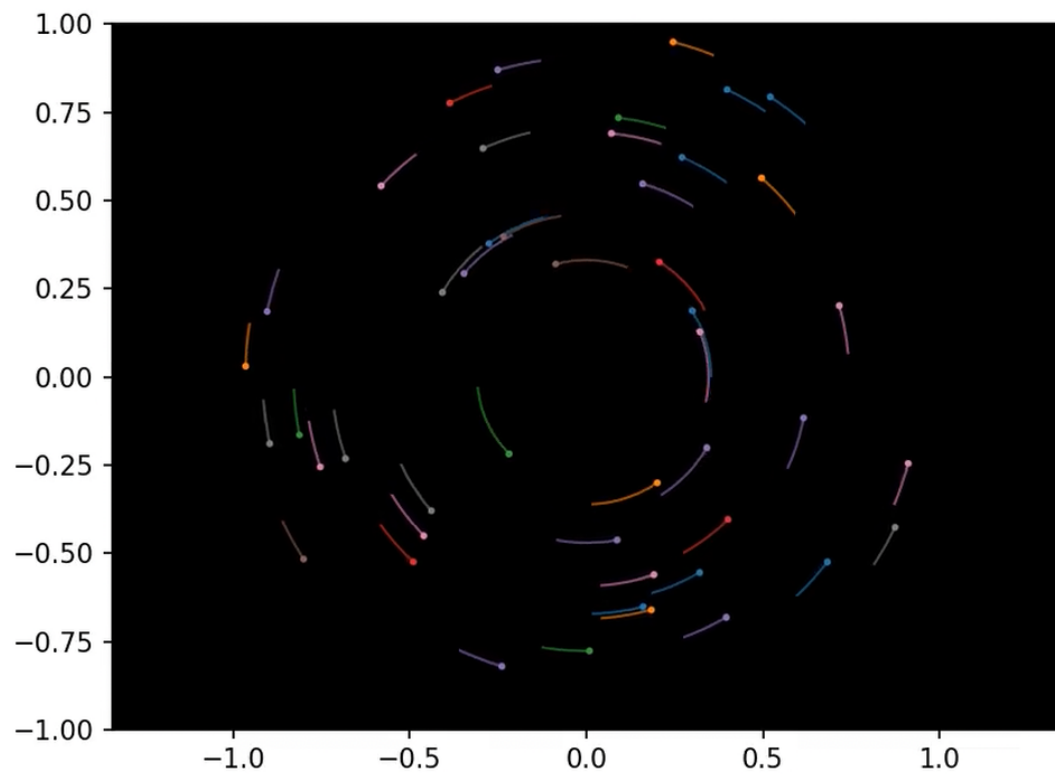


Figura 10: Fotograma de la animación de la evolución temporal del modelo al final de la simulación optimizada. Se observan órbitas circulares, pero no se aprecia bien el disco de acreción.

Asimismo, se calculó la energía media para cada paso del tiempo y se graficó frente al tiempo. Los resultados obtenidos para cada simulación son los siguientes.

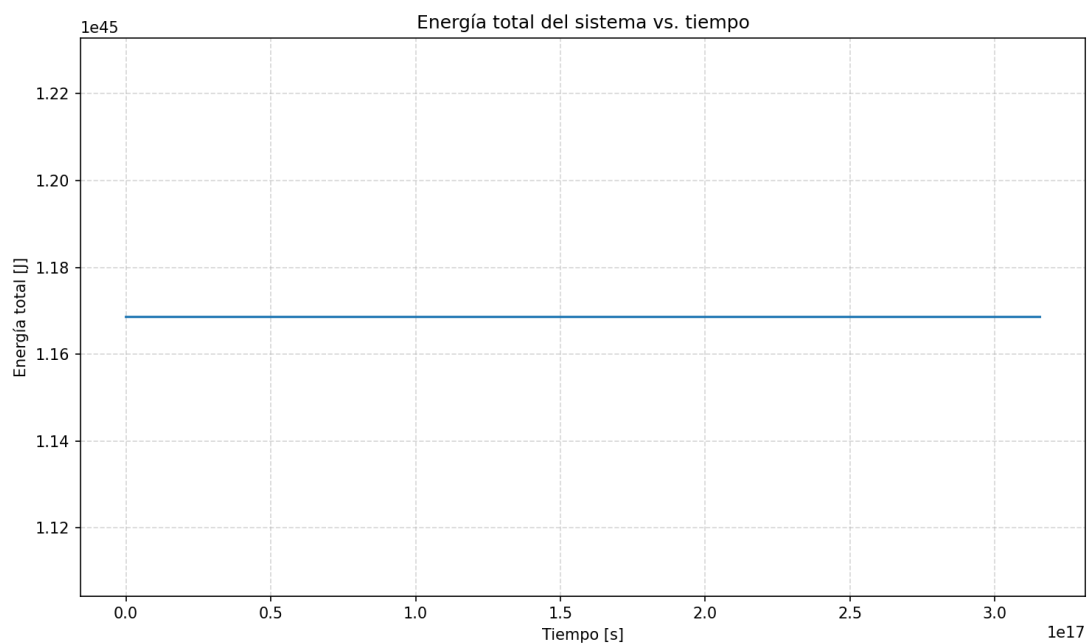


Figura 11: Energía media frente al tiempo del sistema a lo largo de la simulación optimizada en joel.

Volvemos a ver que la energía se mantiene constante en el tiempo y tenemos estado estacionario.

Distribución radial media en función de la distancia.

A continuación, se graficó la distribución radial de masa media en función de la distancia al agujero negro (el centro del sistema).

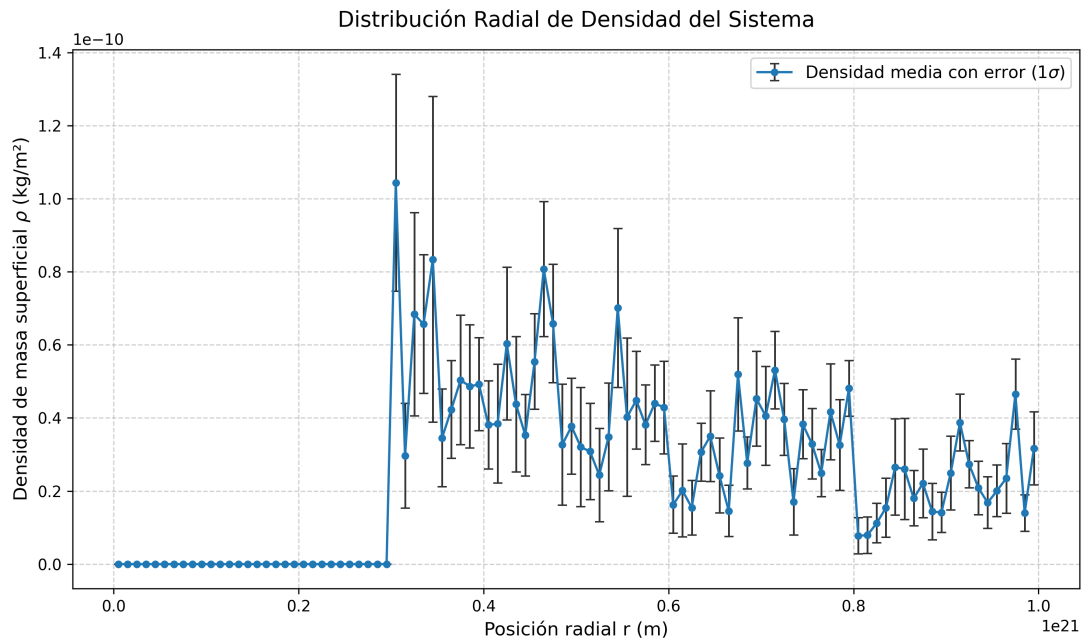


Figura 12: Distribución radial de la densidad de masa superficial en función de la distancia al origen obtenida mediante la simulación optimizada en joel.

En este caso, al haber muchos menos cuerpos, encontramos una caída en la densidad de masa radial con la distancia algo menos pronunciada dado que tenemos muchos menos sistemas que en la simulación anterior. No obstante, sí que encontramos un pico cercano al horizonte de los sucesos (disco de acreción) y a más distancia hay menos densidad de masa.

Momento de inercia medio.

El momento de inercia medio calculado fue $M = (7.8043100 \pm 0.00000012)$, que es solo un orden de magnitud menor que en la simulación de los 100 planetas. A pesar de que la masa total del modelo es mucho menor, como el momento tiene una relación cuadrática con la distancia, es la de los sistemas más alejados la que más contribuye. Por eso, aunque haya 10 veces menos sistemas, se sigue teniendo un momento de inercia enorme porque hay algunos muy alejados del origen.

Flujo de masa absorbido medio.

Nuevamente el flujo de masa absorbido medio es nulo. En este caso, dado que hay una reducción drástica en la densidad de partículas, se producen menos interacciones gravitatorias que desvíen las trayectorias hacia el horizonte de los sucesos.

5. Conclusión.

Como conclusión, observamos que a partir de una configuración inicial determinada, el

sistema formado por un agujero negro y N sistemas solares orbitando a su alrededor es muy estable. Esto lo podemos apreciar en el hecho de que se trata de un sistema estacionario y en el pequeño error en el cálculo del momento de inercia medio.

Hemos comprobado la diferencia entre un sistema de alta densidad de partículas frente a uno de menor densidad, encontrando una menor interacción entre sistemas que resulta en menos masa absorbida por el agujero negro en el de menor densidad. Además, se comprobó que la absorción de un sistema solar es un evento discreto y poco probable. No obstante, se observa que el momento de inercia con 10 veces menos las partículas es únicamente un orden de magnitud menor, lo que se explica por la dependencia cuadrática del momento de inercia con la distancia, que en este modelo toma valores muy grandes.

Asimismo, observamos que la gran parte de la masa de la galaxia se concentra en un disco de acreción cercano al horizonte de los sucesos y que, a más lejanía del origen, se encuentran menos densidad de masa.

Bibliografía.

Agujero negro supermasivo - Wikipedia. *Sistema solar* - Wikipedia. *El Sistema Solar y las Galaxias: Una Introducción a la Dinámica Molecular* - Profesorado de Física Computacional de la UGR.